



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NAME: B.Vishnu

Course Code: CSA 1712

Reg No. 192512410

Course Name: Artificial Intelligence

LAB EXPERIMENTS

1. 8 Puzzle Problem

Aim

To solve the 8-Puzzle problem using the A* Search algorithm in Python.

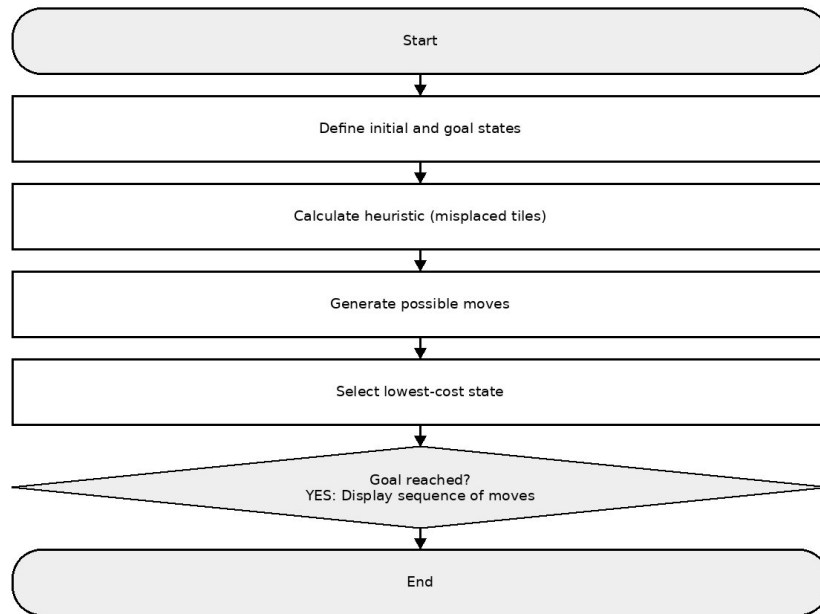
Algorithm

1. Define the initial state and goal state of the puzzle.
2. Calculate the heuristic value (number of misplaced tiles).
3. Generate all possible moves (up, down, left, and right).
4. Select the state with the lowest cost using the A* search algorithm.
5. Repeat the process until the goal state is reached.
6. Display the sequence of moves from the initial state to the goal state.

Flow Chart

1. 8 Puzzle Problem

Flow Chart - A* Search



Program

```
from queue import PriorityQueue

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

def heuristic(state):
    count = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1
    return count

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def copy(state):
    return [row[:] for row in state]

def neighbors(state):
    x, y = find_blank(state)
```

```

moves = [(1,0), (-1,0), (0,1), (0,-1)]
result = []

for dx, dy in moves:
    nx, ny = x+dx, y+dy
    if 0 <= nx < 3 and 0 <= ny < 3:
        new = copy(state)
        new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
        result.append(new)
return result

start = [[1,2,3],
         [4,0,6],
         [7,5,8]]

pq = PriorityQueue()
pq.put((heuristic(start), start))
visited = []

while not pq.empty():
    h, state = pq.get()

    if state == goal:
        print("Goal State Reached")
        for row in state:
            print(row)
        break

    if state not in visited:
        visited.append(state)
        for next_state in neighbors(state):
            if next_state not in visited:
                pq.put((heuristic(next_state), next_state))

```

Input

Initial State

```

1 2 3
4 0 6
7 5 8

```

Output

Goal State Reached

```

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

```

Result

Thus, the 8-Puzzle problem was solved successfully using the *A Search algorithm* in Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
from queue import PriorityQueue

goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

def heuristic(state):
    count = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1
    return count

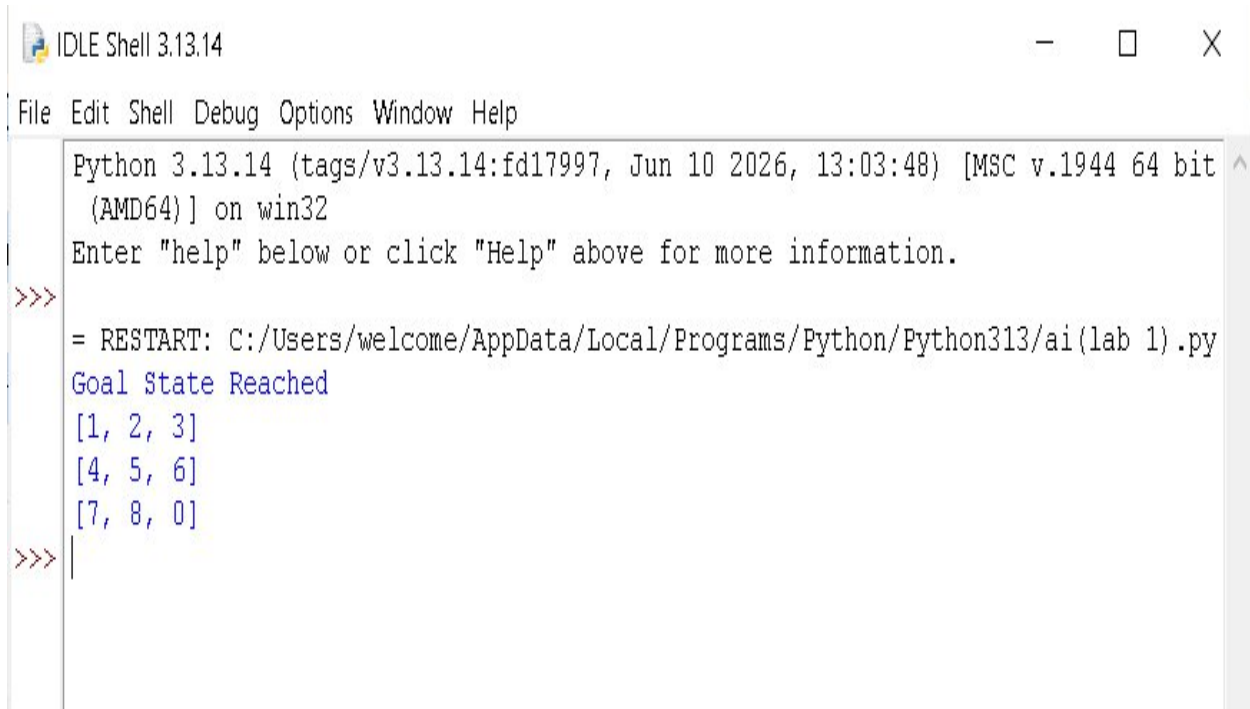
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

def copy(state):
    return [row[:] for row in state]

def neighbors(state):
    x, y = find_blank(state)
    moves = [(1,0), (-1,0), (0,1), (0,-1)]
    result = []

    for dx, dy in moves:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new = copy(state)
            new[x][y], new[nx][ny] = new[nx][ny], new[x][y]
            result.append(new)
    return result

start = [[1,2,3],
         [4,0,6],
         [7,5,8]]
```



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Goal State Reached
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>
```

2. 8 Queen Problem

Aim

To develop a Python program to solve the **8-Queen Problem** using the **Backtracking Algorithm** by placing eight queens on an 8×8 chessboard such that no two queens attack each other.

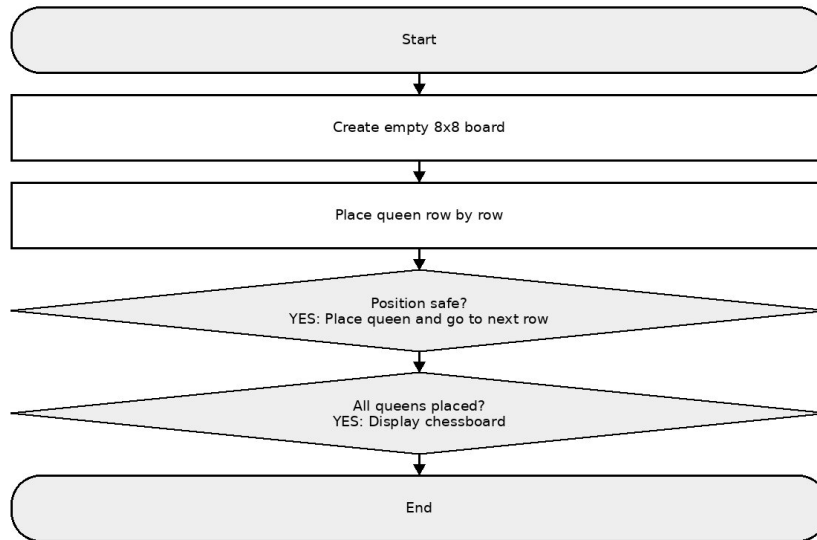
Algorithm

1. Create an empty 8×8 chessboard.
2. Start placing queens from the first row.
3. Check whether the current position is safe:
 - o No queen in the same column.
 - o No queen in the left diagonal.
 - o No queen in the right diagonal.
4. If the position is safe, place the queen.
5. Recursively place queens in the next row.
6. If no safe position exists, backtrack and try another column.
7. Repeat until all eight queens are placed.
8. Display the final chessboard arrangement.

Flow Chart

2. 8 Queen Problem

Flow Chart - Backtracking



Program

```
N = 8

def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col:
            return False
        if abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve(board, row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            if solve(board, row + 1):
                return True
            board[row] = -1
    return False

board = [-1] * N

if solve(board, 0):
    print("Solution:")
    for i in range(N):
```

```

        for j in range(N):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    else:
        print("No Solution Exists")

```

Input

No input is required.

Output

Solution:

```

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . Q . .
. Q . . . . . .
. . . Q . . . .

```

Result

Thus, the **8-Queen Problem** was solved successfully using the **Backtracking Algorithm** in Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
N = 8

def is_safe(board, row, col):
    for i in range(row):
        if board[i] == col:
            return False
        if abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve(board, row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            if solve(board, row + 1):
                return True
            board[row] = -1
    return False

board = [-1] * N

if solve(board, 0):
    print("Solution:")
    for i in range(N):
        for j in range(N):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No Solution Exists")
```

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Solution:
Q . . . . .
. . . . Q . .
. . . . . Q
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
>>> |
```


3. Water Jug Problem

Aim

To write a Python program to solve the **Water Jug Problem** using the **Breadth-First Search (BFS)** algorithm to obtain the required quantity of water.

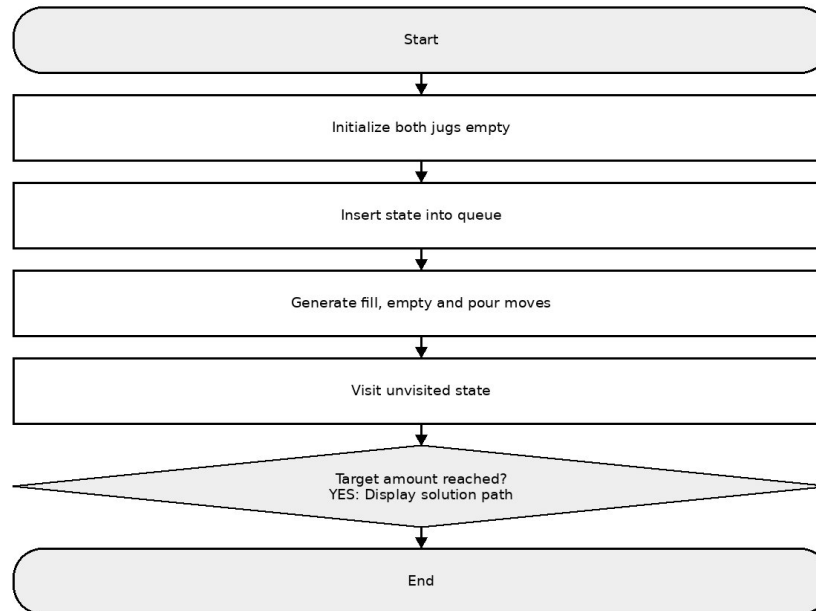
Algorithm

1. Start with both jugs empty.
2. Represent each state as (Jug1, Jug2).
3. Use a queue to store the states to be explored.
4. Generate all possible operations:
 - Fill Jug 1.
 - Fill Jug 2.
 - Empty Jug 1.
 - Empty Jug 2.
 - Pour water from Jug 1 to Jug 2.
 - Pour water from Jug 2 to Jug 1.
5. Mark each visited state to avoid repetition.
6. Repeat until the target amount of water is obtained.
7. Display the sequence of states and the final solution.

Flow Chart

3. Water Jug Problem

Flow Chart - BFS



Program

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue

        visited.add((x, y))
        print((x, y))

        if x == target or y == target:
            print("Target Reached")
            return

        # Fill either jug
        queue.append((jug1, y))
        queue.append((x, jug2))

        # Empty either jug
        queue.append((0, y))
        queue.append((x, 0))

        # Pour Jug1 -> Jug2
```

```
t = min(x, jug2 - y)
queue.append((x - t, y + t))

# Pour Jug2 -> Jug1
t = min(y, jug1 - x)
queue.append((x + t, y - t))

water_jug(4, 3, 2)
```

Input

Capacity of Jug 1 = 4 litres
Capacity of Jug 2 = 3 litres
Target = 2 litres

Output

```
(0, 0)
(4, 0)
(0, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target Reached
```

Result

Thus, the **Water Jug Problem** was implemented successfully in Python, and the required **2 litres** of water was obtained

ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3... - □ ×

File Edit Format Run Options Window Help

```
from collections import deque

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue

        visited.add((x, y))
        print((x, y))

        if x == target or y == target:
            print("Target Reached")
            return

        # Fill either jug
        queue.append((jug1, y))
        queue.append((x, jug2))

        # Empty either jug
        queue.append((0, y))
        queue.append((x, 0))

        # Pour Jug1 -> Jug2
        t = min(x, jug2 - y)
        queue.append((x - t, y + t))

        # Pour Jug2 -> Jug1
        t = min(y, jug1 - x)
        queue.append((x + t, y - t))

water_jug(4, 3, 2)
```

IDLE Shell 3.13.14 - □ ×

File Edit Shell Debug Options Window Help

Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32

Enter "help" below or click "Help" above for more information.

>>>

= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py

(0, 0)

(4, 0)

(0, 3)

(4, 3)

(1, 3)

(3, 0)

(1, 0)

(3, 3)

(0, 1)

(4, 2)

Target Reached

>>>

4. Crypt Arithmetic Problem

Aim

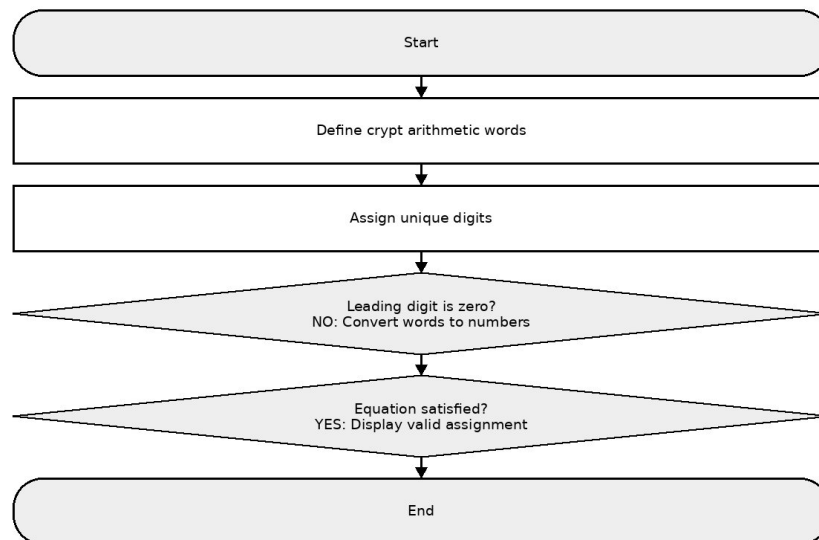
To write a Python program to solve the Crypt Arithmetic Problem by assigning unique digits to letters so that the given arithmetic equation is satisfied.

Algorithm

1. Define the words in the crypt arithmetic problem
2. Assign unique digits (0–9) to each letter.
3. Ensure that the leading letters are not assigned zero.
4. Convert the words into numbers.
5. Check whether the arithmetic equation is satisfied.
6. If a valid assignment is found, display the solution.
7. Stop the program.

Flow Chart

4. Crypt Arithmetic Problem
Flow Chart - Brute-force / permutation search



Program

```
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):
    d = dict(zip(letters, p))

    if d['S'] == 0 or d['M'] == 0:
```

```
        continue

    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']

    if SEND + MORE == MONEY:
        print("Solution:")
        print(d)
        print(SEND, "+", MORE, "=", MONEY)
        break
```

Input

No input is required.

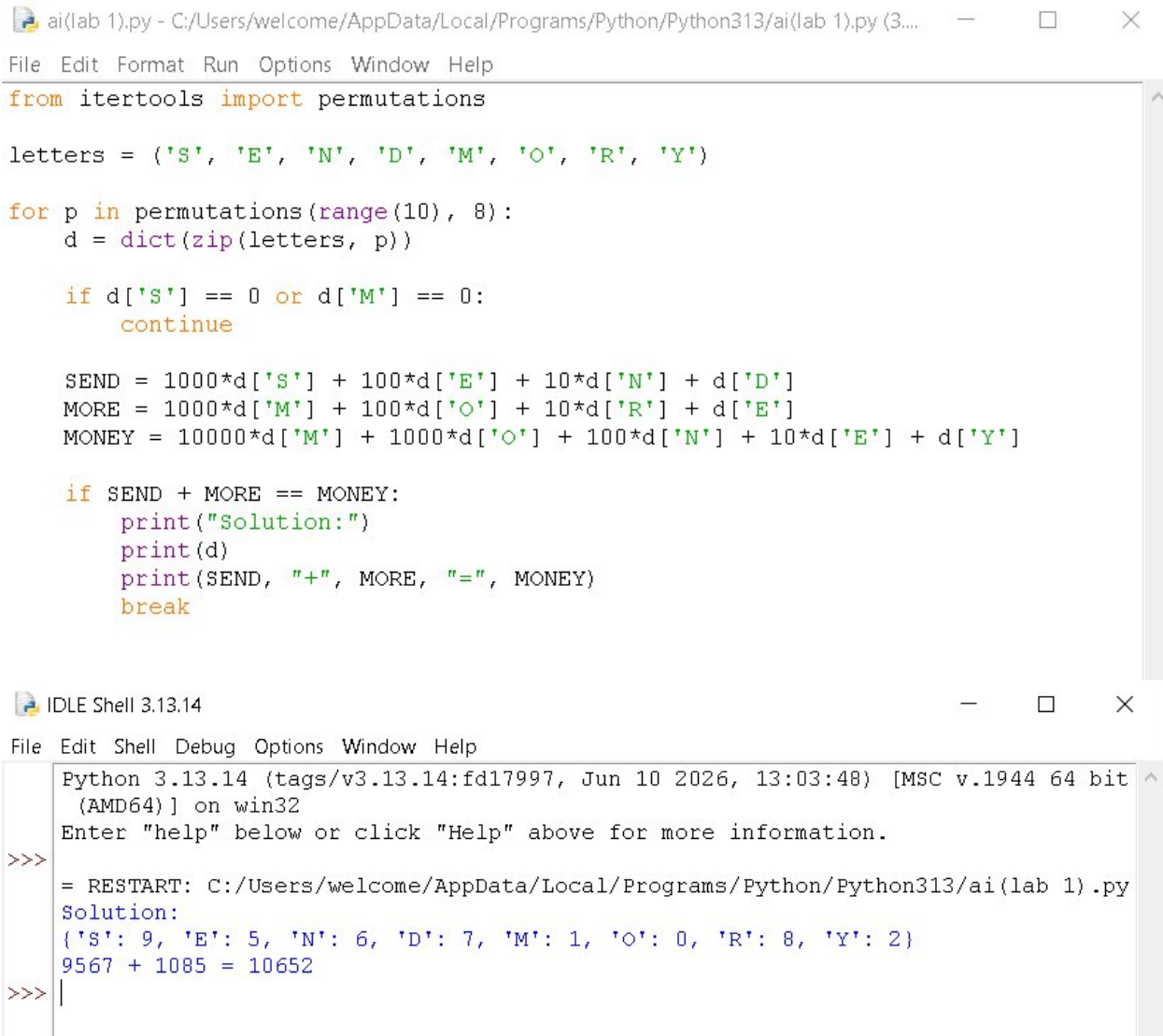
Output

```
Solution:
{'S': 9, 'E': 5, 'N': 6, 'D': 7,
 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
```

```
9567 + 1085 = 10652
```

Result

Thus, the **Crypt Arithmetic Problem** was solved successfully using Python by assigning unique digits to letters that satisfy the given arithmetic expression.



The image shows two windows from a Python IDE. The top window, titled 'ai(lab 1).py', contains a Python script that iterates through permutations of the letters 'S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y' to find a solution to the cryptarithm SEND + MORE = MONEY. The script uses a dictionary to track letter counts and breaks when a valid solution is found. The bottom window, titled 'IDLE Shell 3.13.14', shows the execution output, confirming the solution: {'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2} and the equation 9567 + 1085 = 10652.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
from itertools import permutations

letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')

for p in permutations(range(10), 8):
    d = dict(zip(letters, p))

    if d['S'] == 0 or d['M'] == 0:
        continue

    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']

    if SEND + MORE == MONEY:
        print("Solution:")
        print(d)
        print(SEND, "+", MORE, "=", MONEY)
        break

IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Solution:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
9567 + 1085 = 10652
>>> |
```

5. Missionaries and Cannibals Problem

Aim

To write a Python program to solve the **Missionaries and Cannibals Problem** using **Breadth-First Search (BFS)**.

Algorithm

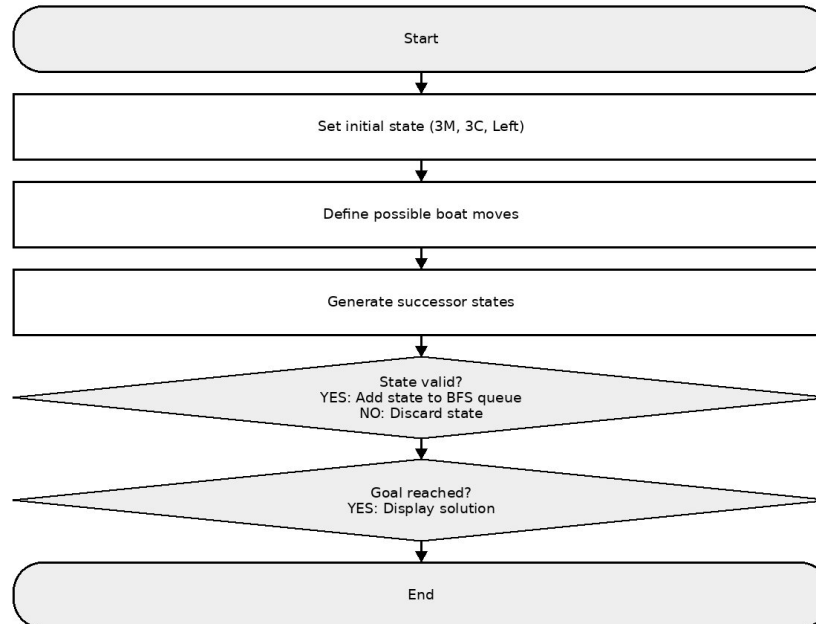
1. Represent the initial state as **(3 Missionaries, 3 Cannibals, Left Bank)**.
2. Represent the goal state as **(0 Missionaries, 0 Cannibals, Right Bank)**.

3. Define all possible boat moves.
4. Generate valid successor states.
5. Ensure missionaries are never outnumbered by cannibals on either bank.
6. Use BFS to explore all valid states.
7. Stop when the goal state is reached and display the solution.

Flow Chart

5. Missionaries and Cannibals Problem

Flow Chart - BFS



Program

```
from collections import deque

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if (m > 0 and m < c):
        return False
    if (3 - m > 0 and 3 - m < 3 - c):
        return False
    return True

def missionaries_cannibals():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')

    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    queue = deque([start])
```



```

visited = set()

while queue:
    state = queue.popleft()

    if state in visited:
        continue

    visited.add(state)
    print(state)
    if state == goal:
        print("Goal Reached")
        return

    m, c, side = state

    for dm, dc in moves:
        if side == 'L':
            nm, nc = m-dm, c-dc
            ns = 'R'
        else:
            nm, nc = m+dm, c+dc
            ns = 'L'

        if is_valid(nm, nc):
            queue.append((nm, nc, ns))

missionaries_cannibals()

```

Input

No input is required.

Output

```

(3, 3, 'L')
(3, 2, 'R')
(3, 1, 'L')
...
(0, 0, 'R')
Goal Reached

```

Result

Thus, the **Missionaries and Cannibals Problem** was solved successfully using the **Breadth-First Search (BFS)** algorithm in Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
from collections import deque

def is_valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False
    if (m > 0 and m < c):
        return False
    if (3 - m > 0 and 3 - m < 3 - c):
        return False
    return True

def missionaries_cannibals():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')

    moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    queue = deque([start])
    visited = set()

    while queue:
        state = queue.popleft()

        if state in visited:
            continue

        visited.add(state)
        print(state)

        if state == goal:
            print("Goal Reached")
            return

        m, c, side = state

        for dm, dc in moves:
            if side == 'L':
                nm, nc = m-dm, c-dc
                ns = 'R'
            else:
                nm, nc = m+dm, c+dc
                ns = 'L'

                if is_valid(nm, nc):
                    queue.append((nm, nc, ns))

Ln: 47 Col: 24
```

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
(3, 3, 'L')
(3, 2, 'R')
(3, 1, 'R')
(2, 2, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 2, 'L')
(0, 0, 'R')
Goal Reached
```

6. Vacuum Cleaner Problem

Aim

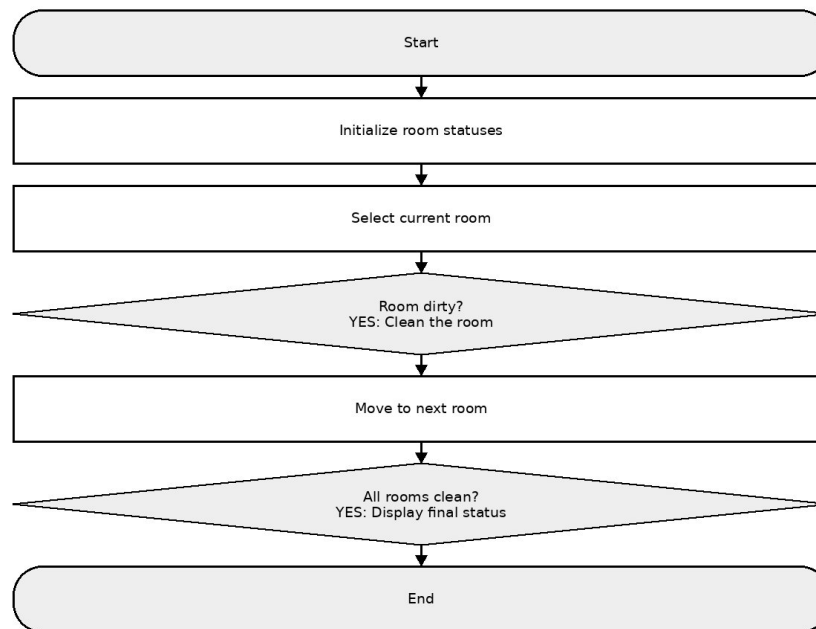
To write a Python program to simulate a Vacuum Cleaner Agent that cleans all dirty rooms.

Algorithm

1. Initialize the rooms and their status (Dirty/Clean).
2. Start from the first room.
3. Check the status of the current room.
4. If the room is dirty, clean it.
5. Move to the next room.
6. Repeat until all rooms are clean.
7. Display the final status of all rooms.

Flow Chart

6. Vacuum Cleaner Problem
Flow Chart - Agent simulation



Program

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}

for room in rooms:
    print("Current Room:", room)

    if rooms[room] == "Dirty":
        print("Cleaning", room)
        rooms[room] = "Clean"

    print(room, ":", rooms[room])

print("\nFinal Room Status:")
for room in rooms:
    print(room, ":", rooms[room])

print("All rooms are clean.")
```

Input

Room A = Dirty
Room B = Dirty

Output

Current Room: A
Cleaning A
A : Clean

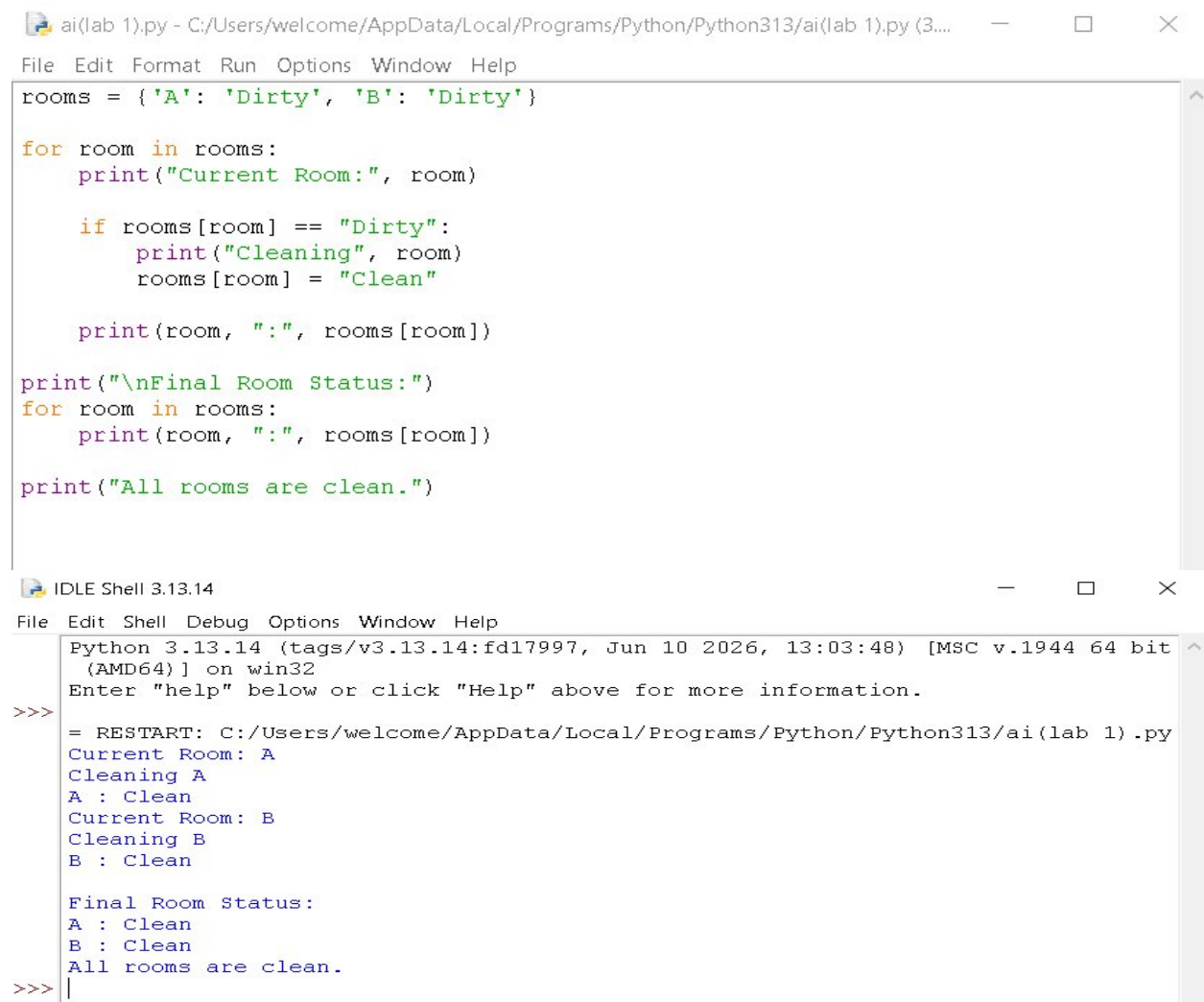
Current Room: B
Cleaning B
B : Clean

Final Room Status:
A : Clean
B : Clean

All rooms are clean.

Result

Thus, the **Vacuum Cleaner Problem** was implemented successfully in Python, and all dirty rooms were cleaned.



The screenshot displays two windows from the Python IDLE 3.13.14 environment. The top window, titled 'ai(lab 1).py', contains the following Python code:

```
rooms = {'A': 'Dirty', 'B': 'Dirty'}

for room in rooms:
    print("Current Room:", room)

    if rooms[room] == "Dirty":
        print("Cleaning", room)
        rooms[room] = "Clean"

    print(room, ":", rooms[room])

print("\nFinal Room Status:")
for room in rooms:
    print(room, ":", rooms[room])

print("All rooms are clean.")
```

The bottom window, titled 'IDLE Shell 3.13.14', shows the output of the program after execution:

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Current Room: A
Cleaning A
A : Clean
Current Room: B
Cleaning B
B : Clean

Final Room Status:
A : Clean
B : Clean
All rooms are clean.
>>> |
```

7. Breadth First Search (BFS)

Aim

To write a Python program to implement the Breadth First Search (BFS) algorithm for graph traversal.

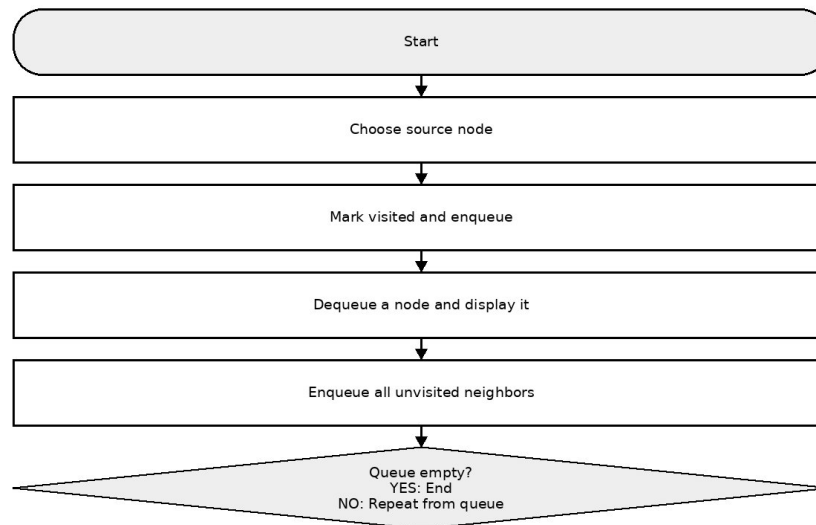
Algorithm

1. Start from the source node.
2. Mark the source node as visited.
3. Insert the source node into a queue.
4. Remove a node from the queue.
5. Display the node.
6. Visit all unvisited adjacent nodes and insert them into the queue.
7. Repeat until the queue becomes empty.

Flow Chart

7. Breadth First Search (BFS)

Flow Chart - Graph traversal



Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```

        'C': ['F'],
        'D': [],
        'E': ['F'],
        'F': []
    }

    visited = set()

    def bfs(start):
        queue = deque([start])
        visited.add(start)

        while queue:
            node = queue.popleft()
            print(node, end=" ")

            for neighbour in graph[node]:
                if neighbour not in visited:
                    visited.add(neighbour)
                    queue.append(neighbour)

    bfs('A')

```

Input

Starting Node:
A

Output

A B C D E F

Result

Thus, the Breadth First Search (BFS) algorithm was implemented successfully.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()

def bfs(start):
    queue = deque([start])
    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

bfs('A')
```

```
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
A B C D E F
>>> |
```

8. Depth First Search (DFS)

Aim

To write a Python program to implement the Depth First Search (DFS) algorithm for graph traversal.

Algorithm

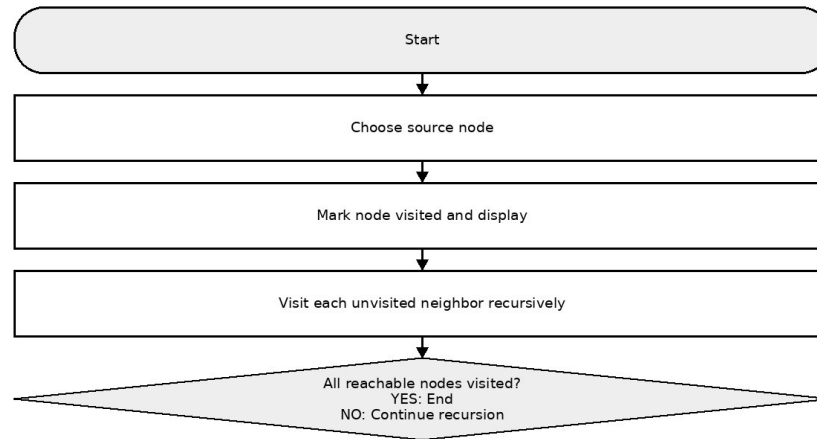
1. Start from the source node.
2. Mark the node as visited.
3. Display the node.

4. Recursively visit all unvisited adjacent nodes.
5. Continue until all nodes are visited.

Flow Chart

8. Depth First Search (DFS)

Flow Chart - Graph traversal



Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

dfs('A')
```

Input

Starting Node:
A

Output

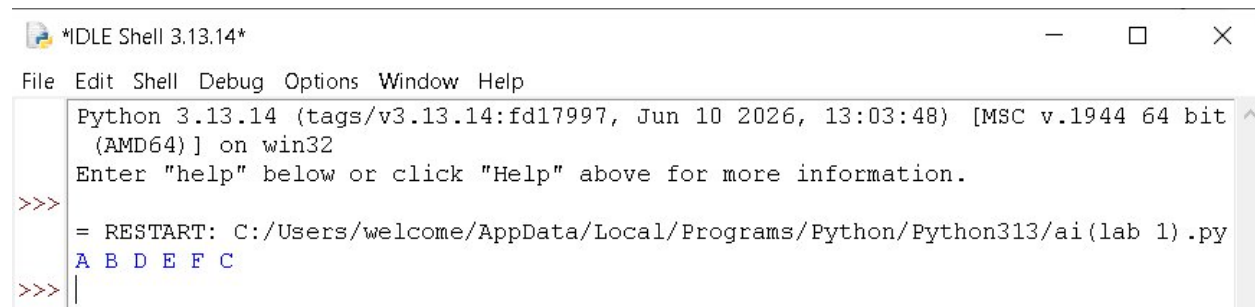
A B D E F C

Result

Thus, the Depth First Search (DFS) algorithm was implemented successfully.



```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....  
File Edit Format Run Options Window Help  
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
    if node not in visited:  
        print(node, end=" ")  
        visited.add(node)  
  
        for neighbour in graph[node]:  
            dfs(neighbour)  
  
dfs('A')
```



```
*IDLE Shell 3.13.14*  
File Edit Shell Debug Options Window Help  
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit  
(AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>  
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py  
A B D E F C  
>>> |
```

9. Travelling Salesman Problem (TSP)

Aim

To write a Python program to solve the Travelling Salesman Problem using brute force.

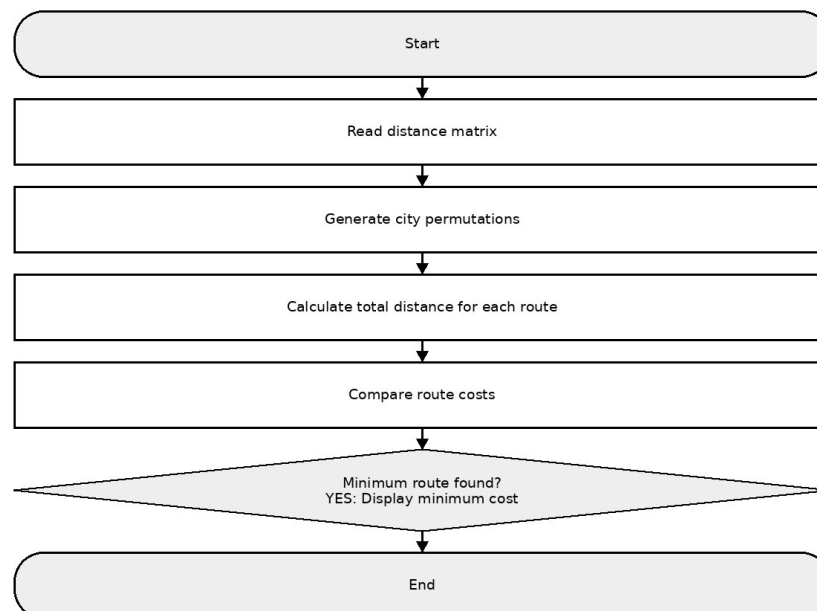
Algorithm

1. Read the distance matrix.
2. Generate all possible city permutations.
3. Calculate the total distance for each route.
4. Select the route with the minimum distance.
5. Display the minimum cost.

Flow Chart

9. Travelling Salesman Problem (TSP)

Flow Chart - Brute force



Program

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
cities = range(1, n)

min_cost = float('inf')

for path in permutations(cities):
    cost = 0
    k = 0

    for city in path:
        cost += graph[k][city]
        k = city

    cost += graph[k][0]

    if cost < min_cost:
        min_cost = cost

print("Minimum Cost:", min_cost)
```

Input

Distance Matrix:

0	10	15	20
10	0	35	25
15	35	0	30
20	25	30	0

Output

Minimum Cost: 80

Result

Thus, the Travelling Salesman Problem was solved successfully using Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
cities = range(1, n)
min_cost = float('inf')

for path in permutations(cities):
    cost = 0
    k = 0

    for city in path:
        cost += graph[k][city]
        k = city

    cost += graph[k][0]

    if cost < min_cost:
        min_cost = cost

print("Minimum Cost:", min_cost)
```

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Minimum Cost: 80
>>> |
```

10. A (A-Star) Algorithm*

Aim

To write a Python program to implement the A* search algorithm.

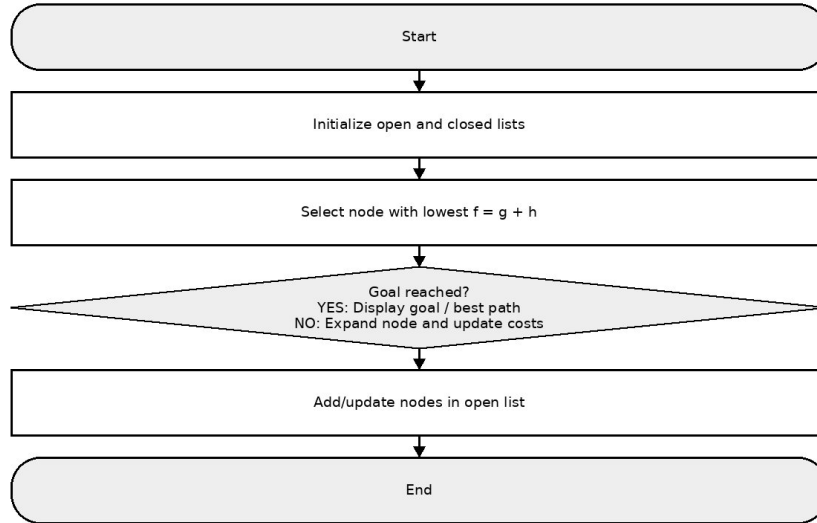
Algorithm

1. Initialize the open list with the start node.
2. Maintain a closed list for visited nodes.
3. Select the node with the lowest $f = g + h$ value.
4. If the goal node is reached, stop.
5. Otherwise, expand the node and update the costs.
6. Repeat until the goal is found.

Flow Chart

10. A (A-Star) Algorithm*

Flow Chart - A* Search



Program

```
from queue import PriorityQueue

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 6)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 0
}

def astar(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    cost = {start: 0}

    while not pq.empty():
        _, node = pq.get()

        if node == goal:
            print("Goal Reached")
```

```
        return

    for neighbour, weight in graph[node]:
        new_cost = cost[node] + weight

        if neighbour not in cost or new_cost < cost[neighbour]:
            cost[neighbour] = new_cost
            priority = new_cost + heuristic[neighbour]
            pq.put((priority, neighbour))

    astar('A', 'F')
```

Input

Start Node:
A

Goal Node:
F

Output

Goal Reached

Result

Thus, the A* search algorithm was implemented successfully in Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help

graph = {
    'A': [(('B', 1), ('C', 3))],
    'B': [(('D', 3), ('E', 6))],
    'C': [(('F', 5))],
    'D': [],
    'E': [(('F', 2))],
    'F': []
}

heuristic = {
    'A': 6,
    'B': 4,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 0
}

def astar(start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    cost = {start: 0}

    while not pq.empty():
        _, node = pq.get()

        if node == goal:
            print("Goal Reached")
            return

        for neighbour, weight in graph[node]:
            new_cost = cost[node] + weight

            if neighbour not in cost or new_cost < cost[neighbour]:
                cost[neighbour] = new_cost
                priority = new_cost + heuristic[neighbour]
                pq.put((priority, neighbour))

astar('A', 'F')
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Goal Reached
>>>
```


11. Map Coloring using CSP

Aim

To write a Python program to solve the **Map Coloring Problem** using **Constraint Satisfaction Problem (CSP)**.

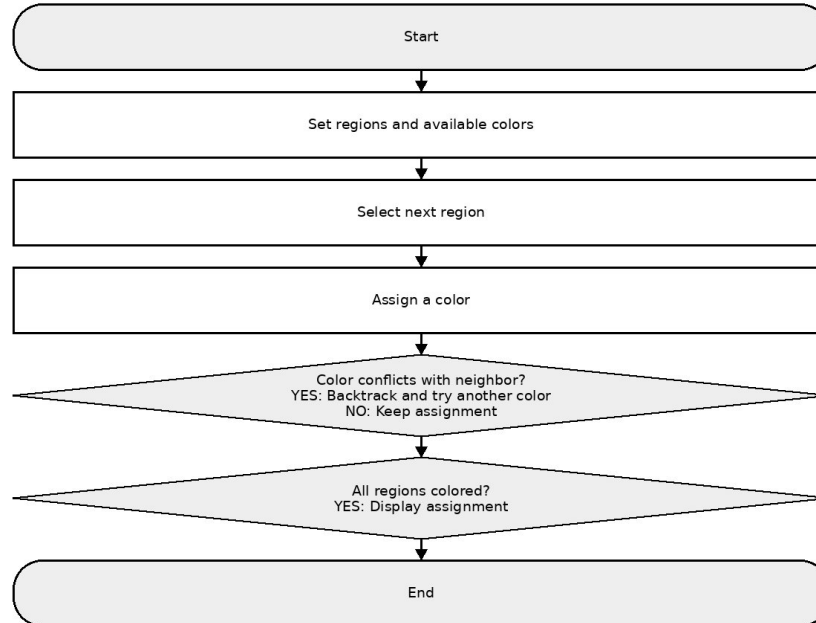
Algorithm

1. Start with a set of regions and available colors.
2. Represent each region as a variable.
3. Assign a color to each region.
4. Check whether the assigned color conflicts with neighboring regions.
5. If there is no conflict, move to the next region.
6. If a conflict occurs, backtrack and try another color.
7. Repeat until all regions are colored.
8. Display the final color assignment.

Flow Chart

11. Map Coloring using CSP

Flow Chart - CSP backtracking



Program

```
def map_coloring():  
    regions = ['A', 'B', 'C', 'D']
```

```

neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
assignment = {}

def is_valid(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

def solve(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_valid(region, color):
            assignment[region] = color
            if solve(index + 1):
                return True
            del assignment[region]
    return False

if solve(0):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution found")

map_coloring()

```

Input

Regions = A, B, C, D

Colors = Red, Green, Blue

Output

Map Coloring Solution:

A : Red

B : Green

C : Blue

D : Red

Result

Thus, the **Map Coloring Problem** was implemented successfully in Python using the **CSP backtracking algorithm**.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....
File Edit Format Run Options Window Help
'A': ['B', 'C'],
'B': ['A', 'C', 'D'],
'C': ['A', 'B', 'D'],
'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
assignment = {}

def is_valid(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True

def solve(index):
    if index == len(regions):
        return True

    region = regions[index]

    for color in colors:
        if is_valid(region, color):
            assignment[region] = color

            if solve(index + 1):
                return True

            del assignment[region]

    return False

if solve(0):
    print("Map Coloring Solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution found")

map_coloring()
```

```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
>>>
```

12. Tic Tac Toe Game

Aim

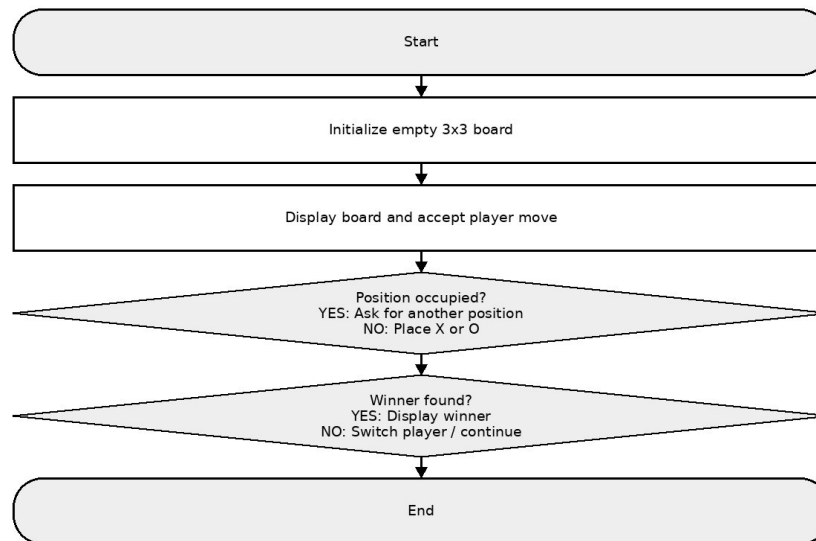
To write a Python program to implement a simple **Tic Tac Toe game**.

Algorithm

1. Start with an empty 3×3 board.
2. Display the board.
3. Allow Player X to enter a position.

Flow Chart

12. Tic Tac Toe Game
Flow Chart - Game simulation



Program

```
def display(board):  
    print(board[0], "|", board[1], "|", board[2])  
    print("---+---+---")  
    print(board[3], "|", board[4], "|", board[5])  
    print("---+---+---")  
    print(board[6], "|", board[7], "|", board[8])  
  
def check_winner(board, player):  
    wins = [  

```

```
    [0, 1, 2],
    [3, 4, 5],
    [6, 7, 8],
    [0, 3, 6],
    [1, 4, 7],
    [2, 5, 8],
    [0, 4, 8],
    [2, 4, 6]
]
```

```
for w in wins:
    if all(board[i] == player for i in w):
        return True
    return False
board = ['1', '2', '3', '4', '5', '6', '7', '8', '9']
player = 'X'
for turn in range(9):
    display(board)
    pos = int(input("Enter position: ")) - 1
    if board[pos] in ['X', 'O']:
        print("Position already occupied")
        continue
    board[pos] = player
    if check_winner(board, player):
        display(board)
        print("Player", player, "wins")
        break
    player = 'O' if player == 'X' else 'X'
else:
    display(board)
    print("Game Draw")
```

Input

Enter position: 1

Enter position: 4

Enter position: 2

Enter position: 5

Enter position: 3

Output

```
X | X | X
```

```
--+---+--
```

```
O | O | 6
```

```
--+---+--
```

```
7 | 8 | 9
```

Player X wins

Result

Thus, the **Tic Tac Toe game** was implemented successfully in Python.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
def display(board):
    print(board[0], "|", board[1], "|", board[2])
    print("---+---+---")
    print(board[3], "|", board[4], "|", board[5])
    print("---+---+---")
    print(board[6], "|", board[7], "|", board[8])

def check_winner(board, player):
    wins = [
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8],
        [0, 3, 6],
        [1, 4, 7],
        [2, 5, 8],
        [0, 4, 8],
        [2, 4, 6]
    ]

    for w in wins:
        if all(board[i] == player for i in w):
            return True

    return False

board = ['1', '2', '3', '4', '5', '6', '7', '8', '9']
player = 'X'

for turn in range(9):
    display(board)

    pos = int(input("Enter position: ")) - 1

    if board[pos] in ['X', 'O']:
        print("Position already occupied")
        continue

    board[pos] = player

*IDLE Shell 3.13.14*
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9
Enter position:
```

13. Minimax Algorithm for Gaming

Aim

To write a Python program to implement the **Minimax Algorithm** for gaming.

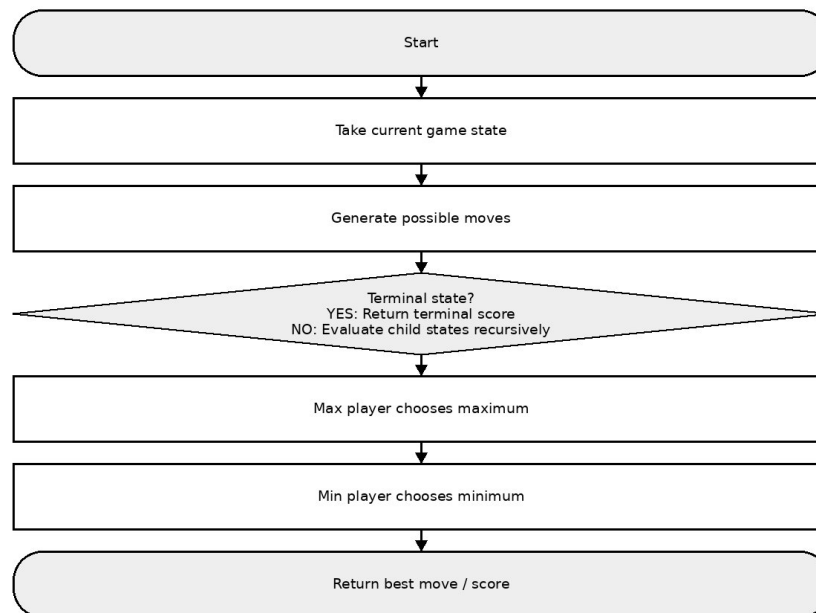
Algorithm

1. Start with the current game state.
2. Generate all possible moves.
3. Check whether the state is a terminal state.
4. Assign a score to the terminal state.
5. The maximizing player selects the maximum score.
6. The minimizing player selects the minimum score.
7. Continue recursively until the best move is found.
8. Display the best move.

Flow Chart

13. Minimax Algorithm for Gaming

Flow Chart - Minimax



Program

```
def minimax(depth, is_max):  
    scores = [10, 5, -10, 0]  
    if depth == 0:  
        return scores[0]
```

```
    if is_max:
        best = -1000
        for i in range(2):
            value = minimax(depth - 1, False)
            best = max(best, value)
        return best
    else:
        best = 1000
        for i in range(2):
            value = minimax(depth - 1, True)
            best = min(best, value)
        return best

print("Best Score:", minimax(3, True))
```

Input

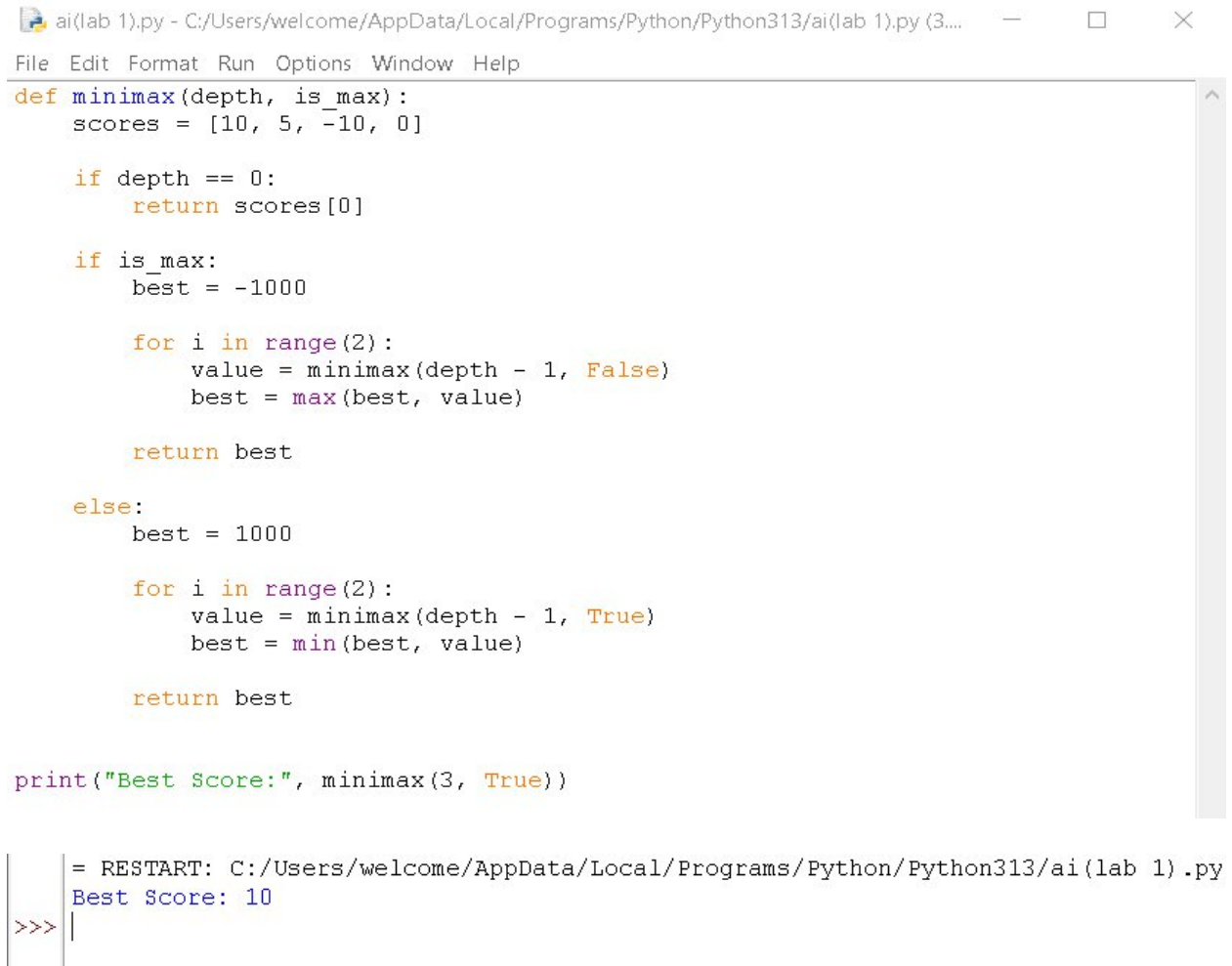
Game depth = 3

Output

Best Score: 10

Result

Thus, the **Minimax Algorithm** was implemented successfully in Python for gaming.



The screenshot shows a Python IDE window titled "ai(lab 1).py". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code defines a minimax function that takes depth and is_max as arguments. It initializes scores to [10, 5, -10, 0]. If depth is 0, it returns scores[0]. If is_max is True, it sets best to -1000 and iterates over the scores list, updating best with the maximum value. If is_max is False, it sets best to 1000 and iterates over the scores list, updating best with the minimum value. The function is called with minimax(3, True), and the output is "Best Score: 10".

```
def minimax(depth, is_max):
    scores = [10, 5, -10, 0]

    if depth == 0:
        return scores[0]

    if is_max:
        best = -1000
        for i in range(2):
            value = minimax(depth - 1, False)
            best = max(best, value)
        return best
    else:
        best = 1000
        for i in range(2):
            value = minimax(depth - 1, True)
            best = min(best, value)
        return best

print("Best Score:", minimax(3, True))
```

```
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Best Score: 10
>>>
```

14. Alpha-Beta Pruning Algorithm for Gaming

Aim

To write a Python program to implement the **Alpha-Beta Pruning Algorithm** for gaming.

Algorithm

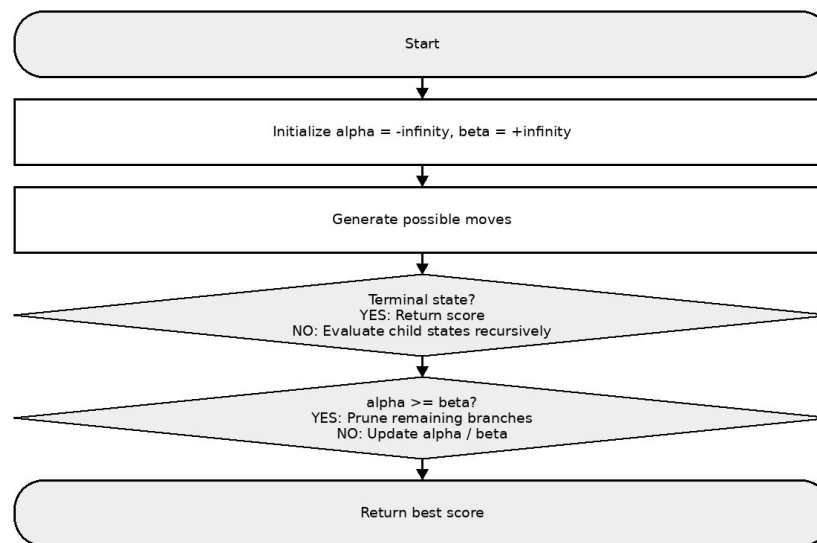
1. Start with the game tree.

2. Set $\alpha = -\infty$ and $\beta = +\infty$.
3. Generate possible moves.
4. Find the maximum value for the maximizing player.
5. Find the minimum value for the minimizing player.
6. Update α and β values.
7. If α becomes greater than or equal to β , prune the remaining branches.
8. Continue until the best move is obtained.
9. Display the best score.

Flow Chart

14. Alpha-Beta Pruning Algorithm for Gaming

Flow Chart - Alpha-Beta Pruning



Program

```

def alphabeta(depth, alpha, beta, is_max):
    if depth == 0:
        return 10
    if is_max:
        best = -1000
        for i in range(2):
            value = alphabeta(depth - 1, alpha, beta, False)
            best = max(best, value)
            alpha = max(alpha, best)
            if alpha >= beta:

```

```
        break
    return best
else:
    best = 1000

    for i in range(2):
        value = alphabeta(depth - 1, alpha, beta, True)
        best = min(best, value)
        beta = min(beta, best)
        if alpha >= beta:
            break
    return best
print("Best Score:",
      alphabeta(3, -1000, 1000, True))
```

Input

Game depth = 3

Alpha = -1000

Beta = 1000

Output

Best Score: 10

Result

Thus, the **Alpha-Beta Pruning Algorithm** was implemented successfully in Python for gaming.

```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
def alphabeta(depth, alpha, beta, is_max):
    if depth == 0:
        return 10

    if is_max:
        best = -1000
        for i in range(2):
            value = alphabeta(depth - 1, alpha, beta, False)
            best = max(best, value)
            alpha = max(alpha, best)
            if alpha >= beta:
                break
        return best
    else:
        best = 1000
        for i in range(2):
            value = alphabeta(depth - 1, alpha, beta, True)
            best = min(best, value)
            beta = min(beta, best)
            if alpha >= beta:
                break
        return best

print("Best Score:",
      alphabeta(3, -1000, 1000, True))

= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Best Score: 10
>>> |
```

15. Decision Tree

Aim

To write a Python program to implement a **Decision Tree** for classification.

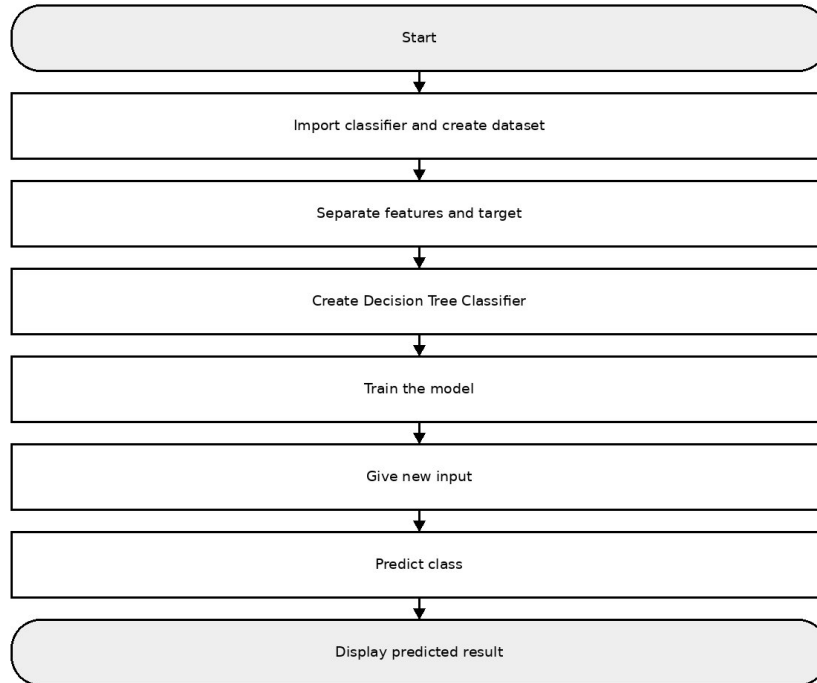
Algorithm

9. Import the required libraries.
10. Create the input dataset.
11. Separate the features and target values.
12. Create a Decision Tree Classifier.
13. Train the model using the dataset.
14. Give a new input value to the model.
15. Predict the class.
16. Display the predicted result.

Flow Chart

15. Decision Tree

Flow Chart - Machine learning



Program

```
from sklearn.tree import DecisionTreeClassifier

# Input data
X = [
    [20, 1],
    [25, 1],
    [30, 0],
    [35, 0],
    [40, 0],
    [45, 1]
]

# Target data
y = [0, 0, 1, 1, 1, 1]

# Create Decision Tree
model = DecisionTreeClassifier()

# Train the model
```

```
model.fit(X, y)

# Predict for new data
age = int(input("Enter age: "))
family_history = int(input("Family history (1-Yes, 0-No): "))
prediction = model.predict([[age, family_history]])
print("Predicted Class:", prediction[0])
```

Input

Enter age: 30

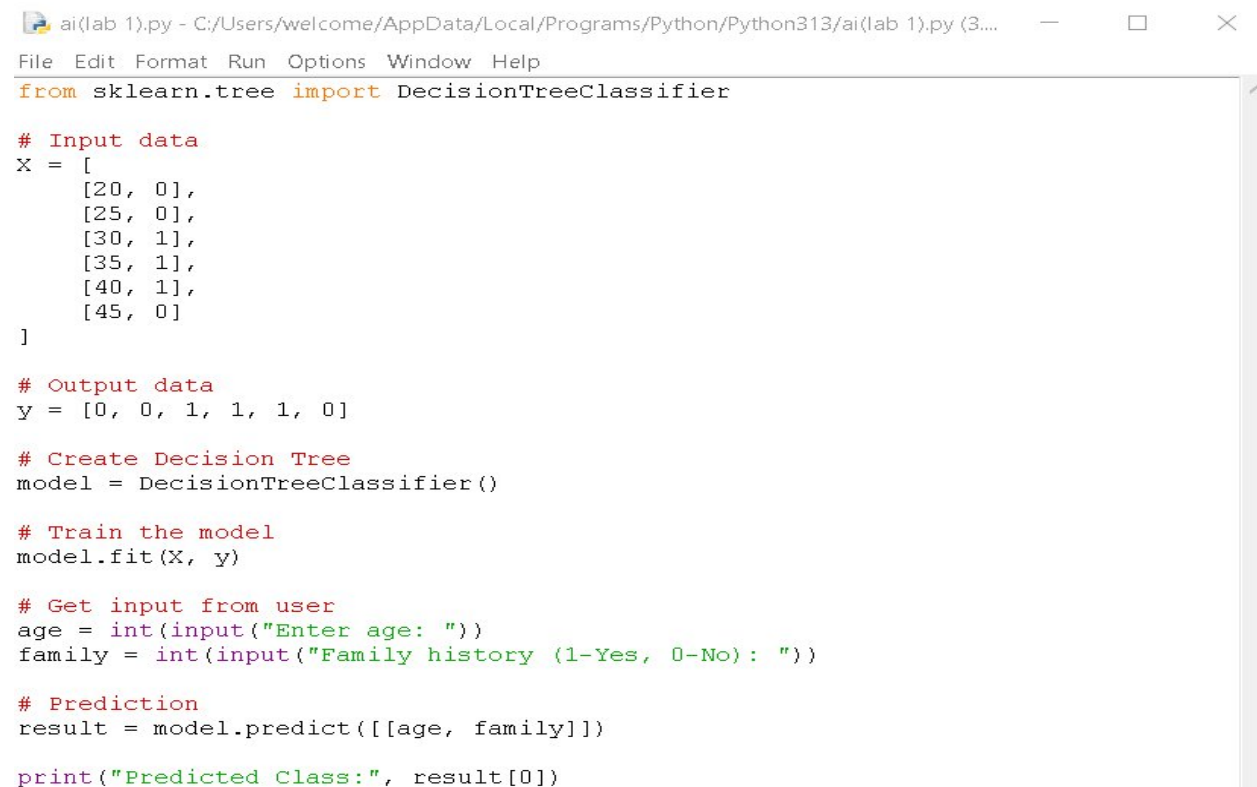
Family history (1-Yes, 0-No): 0

Output

Predicted Class: 1

Result

Thus, the **Decision Tree classification algorithm** was implemented successfully in Python.



The screenshot shows a Python IDE window titled "ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3....". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
from sklearn.tree import DecisionTreeClassifier

# Input data
X = [
    [20, 0],
    [25, 0],
    [30, 1],
    [35, 1],
    [40, 1],
    [45, 0]
]

# Output data
y = [0, 0, 1, 1, 1, 0]

# Create Decision Tree
model = DecisionTreeClassifier()

# Train the model
model.fit(X, y)

# Get input from user
age = int(input("Enter age: "))
family = int(input("Family history (1-Yes, 0-No): "))

# Prediction
result = model.predict([[age, family]])

print("Predicted Class:", result[0])
```



```
>>> = RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1) .py
Best Score: 10
```

16. Feed Forward Neural Network

Aim

To write a Python program to implement a **Feed Forward Neural Network** for classification.

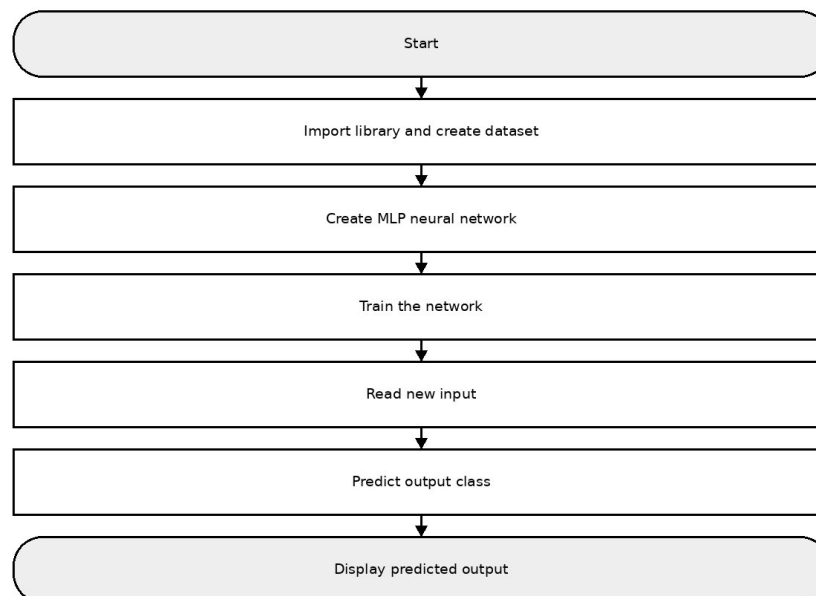
Algorithm

17. Import the required libraries.
18. Create the input dataset and target values.
19. Create a Feed Forward Neural Network using MLPClassifier.
20. Train the network using the training data.
21. Give a new input to the trained network.
22. Predict the output class.
23. Display the predicted result.

Flow Chart

16. Feed Forward Neural Network

Flow Chart - Neural network



Program

```
from sklearn.neural_network import MLPClassifier
```

```
# Input data
X = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]

# Target data
y = [0, 1, 1, 0]

# Create Feed Forward Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(4,),
    max_iter=2000,
    random_state=1
)

# Train the network
model.fit(X, y)

# Get input
a = int(input("Enter first value (0 or 1): "))
b = int(input("Enter second value (0 or 1): "))

# Predict output
prediction = model.predict([[a, b]])
print("Predicted Output:", prediction[0])
```

Input

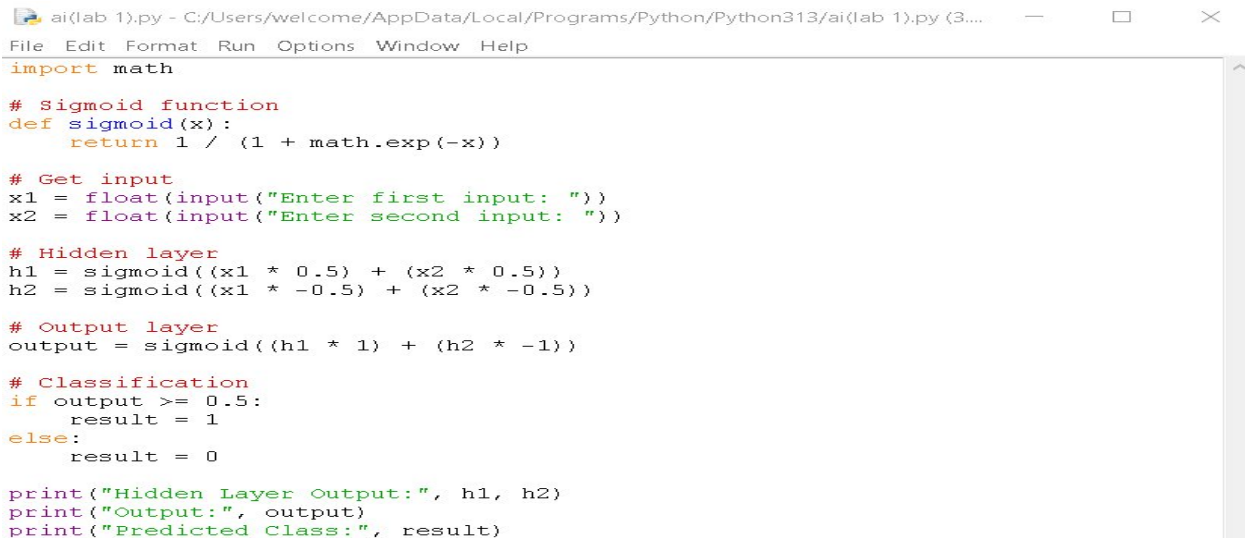
```
Enter first value (0 or 1): 1
Enter second value (0 or 1): 0
```

Output

```
Predicted Output: 1
```

Result

Thus, the **Feed Forward Neural Network** was implemented successfully in Python for classification.



```
ai(lab 1).py - C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py (3...
File Edit Format Run Options Window Help
import math

# Sigmoid function
def sigmoid(x):
    return 1 / (1 + math.exp(-x))

# Get input
x1 = float(input("Enter first input: "))
x2 = float(input("Enter second input: "))

# Hidden layer
h1 = sigmoid((x1 * 0.5) + (x2 * 0.5))
h2 = sigmoid((x1 * -0.5) + (x2 * -0.5))

# Output layer
output = sigmoid((h1 * 1) + (h2 * -1))

# Classification
if output >= 0.5:
    result = 1
else:
    result = 0

print("Hidden Layer Output:", h1, h2)
print("Output:", output)
print("Predicted Class:", result)
```

```
= RESTART: C:/Users/welcome/AppData/Local/Programs/Python/Python313/ai(lab 1).py
Enter first input: 20
Enter second input: 4
Hidden Layer Output: 0.9999938558253978 6.144174602214718e-06
Output: 0.7310561625870515
Predicted Class: 1
```

17: Sum of Integers from 1 to N

Aim

To write and execute a Prolog program to find the sum of integers from 1 to N.

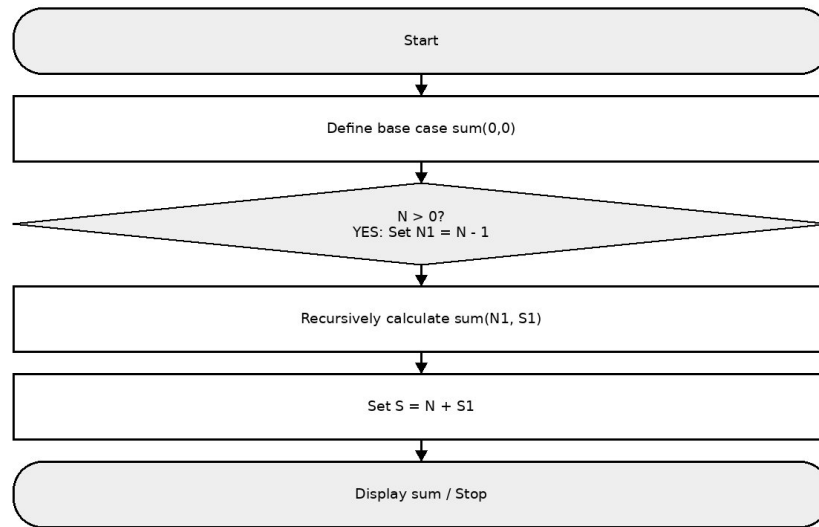
Algorithm

1. Start the program.
2. Define the base case sum(0,0).
3. Check whether N is greater than 0.
4. Decrease N by 1.
5. Recursively calculate the sum.
6. Add N to the recursive sum.
7. Display the result.
8. Stop.

Flow Chart

17: Sum of Integers from 1 to N

Flow Chart - Recursion in Prolog



Program

sum(0, 0).

sum(N, S) :-

 N > 0,

 N1 is N - 1,

 sum(N1, S1),

 S is N + S1.

Output

?- sum(5, S).

S = 15.

Result

Thus, the Prolog program was successfully executed to calculate the sum of integers from 1 to N.

```
19 ?- [experiment17].  
true.
```

```
20 ?- sum(5, S).  
S = 15
```

18: Prolog Program for Database with NAME and DOB

Aim

To write and execute a Prolog program to create a database containing names and dates of birth and retrieve the required information.

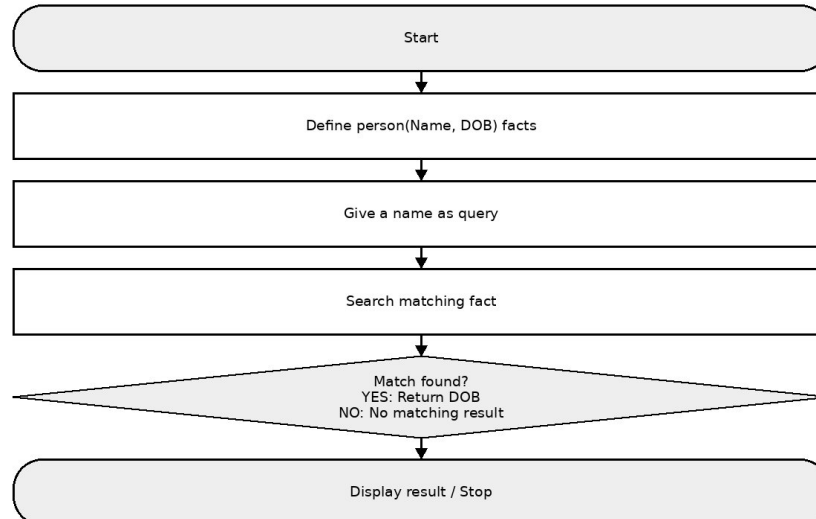
Algorithm

1. Start the program.
2. Define facts containing a person's name and date of birth.
3. Store the facts using the predicate `person(Name, DOB)`.
4. Use a query to search for a person's DOB.
5. Display the matching result.
6. Stop.

Flow Chart

18: Prolog Program for Database with NAME and DOB

Flow Chart - Prolog database



Program

```
person(ragul, '10-05-2005').
```

```
person(rahul, '15-08-2004').
```

```
person(anu, '20-12-2005').
```

```
person(kavin, '05-03-2004').
```

```
find_dob(Name, DOB) :-
```

```
    person(Name, DOB).
```

Output

```
?- find_dob(ragul, DOB).
```

```
DOB = '10-05-2005'.
```

```
?- find_dob(anu, DOB).
```

```
DOB = '20-12-2005'.
```

Result

Thus, the Prolog database was successfully created and the date of birth was retrieved using queries.

A screenshot of the SWI-Prolog graphical user interface (GUI) window. The title bar reads "SWI-Prolog -- C:/Users/welcome/Desktop/experiment18.pl". The menu bar includes "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content: a welcome message for SWI-Prolog (version 10.1.13), a disclaimer about the warranty, and instructions for online help and built-in help. Below this, three Prolog queries are entered and executed: 19 ?- [experiment18]., 20 ?- person(ragul, DOB)., and 21 ?- person(anu, DOB).. The results of the queries are shown below each query: "true." for the first, and "DOB = '10-05-2005'." and "DOB = '20-12-2005'." for the second and third respectively. A vertical scrollbar is visible on the right side of the text area.

```
SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

19 ?- [experiment18].
true.

20 ?- person(ragul, DOB).
DOB = '10-05-2005'.

21 ?- person(anu, DOB).
DOB = '20-12-2005'.
```

19: Prolog Program for Student-Teacher-Subject Code

Aim

To write and execute a Prolog program to represent the relationship between students, teachers, and subject codes.

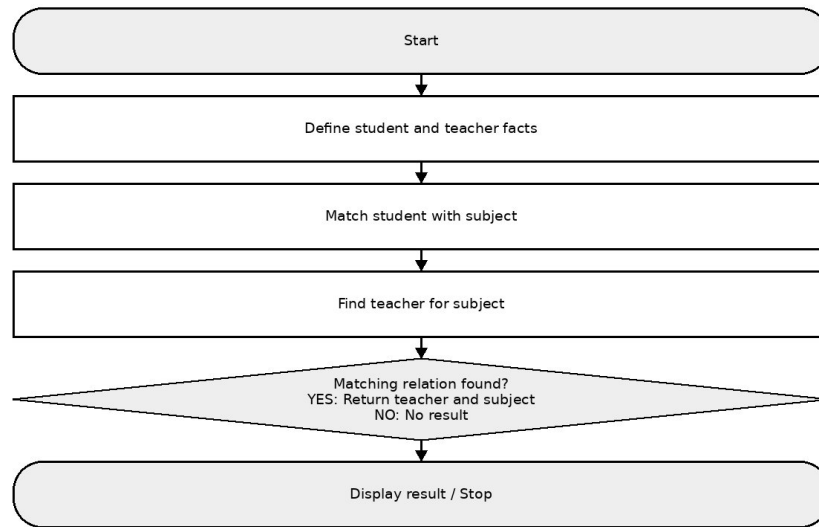
Algorithm

1. Start the program.
2. Define facts for students and their subject codes.
3. Define facts for teachers and the subjects they handle.
4. Create a rule to find the teacher for a student's subject.
5. Execute the required query.
6. Display the result.
7. Stop.

Flow Chart

19: Prolog Program for Student-Teacher-Subject Code

Flow Chart - Prolog facts and rules



Program

student(ragul, cs101).

student(rahul, cs102).

student(anu, cs103).

teacher(john, cs101).

teacher(priya, cs102).

teacher(kumar, cs103).

student_teacher(Student, Teacher, Subject) :-

student(Student, Subject),

teacher(Teacher, Subject).

Output

?- student_teacher(ragul, Teacher, Subject).

Teacher = john,

Subject = cs101.

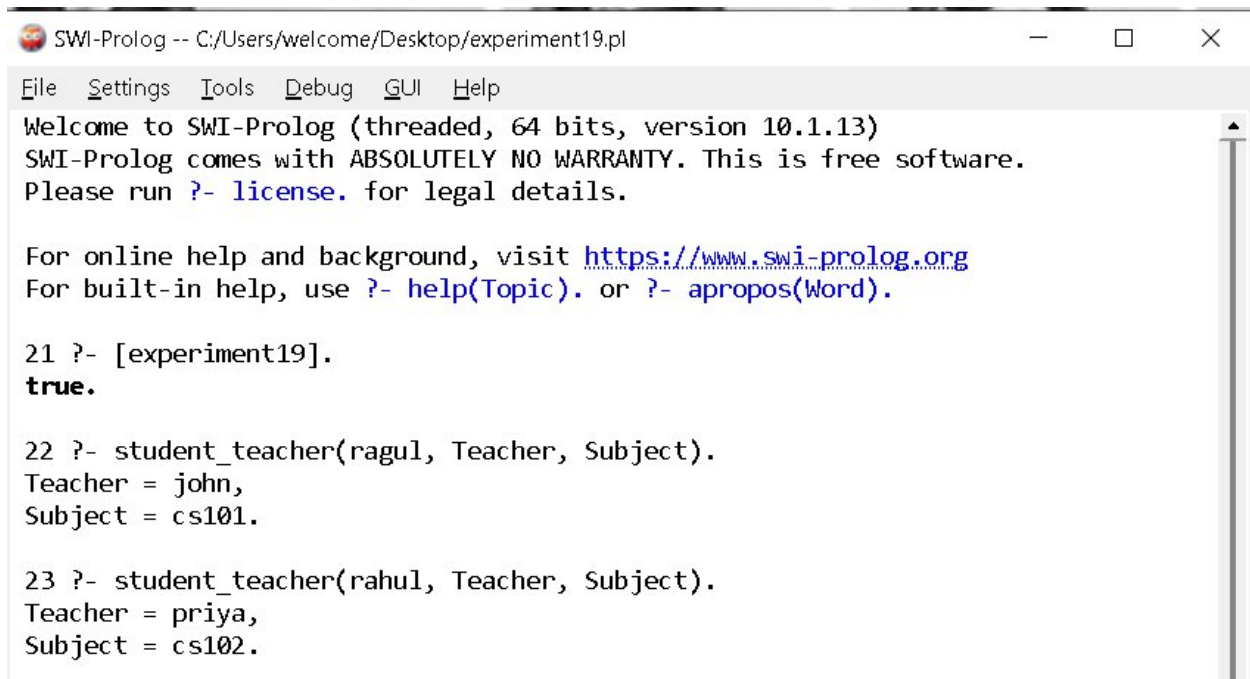
?- student_teacher(rahul, Teacher, Subject).

Teacher = priya,

Subject = cs102.

Result

Thus, the Prolog program was successfully executed to find the teacher and subject code associated with a student.

A screenshot of the SWI-Prolog graphical user interface. The window title is "SWI-Prolog -- C:/Users/welcome/Desktop/experiment19.pl". The menu bar includes "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content: a welcome message for version 10.1.13, a disclaimer about warranty, a link to the SWI-Prolog website, and two Prolog queries. The first query, "?- [experiment19].", returns "true.". The second query, "?- student_teacher(ragul, Teacher, Subject).", returns "Teacher = john, Subject = cs101.". The third query, "?- student_teacher(rahul, Teacher, Subject).", returns "Teacher = priya, Subject = cs102.". A vertical scrollbar is visible on the right side of the text area.

```
SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

21 ?- [experiment19].
true.

22 ?- student_teacher(ragul, Teacher, Subject).
Teacher = john,
Subject = cs101.

23 ?- student_teacher(rahul, Teacher, Subject).
Teacher = priya,
Subject = cs102.
```

20: Prolog Program for Planets Database

Aim

To write and execute a Prolog program to create a database of planets and retrieve planet information.

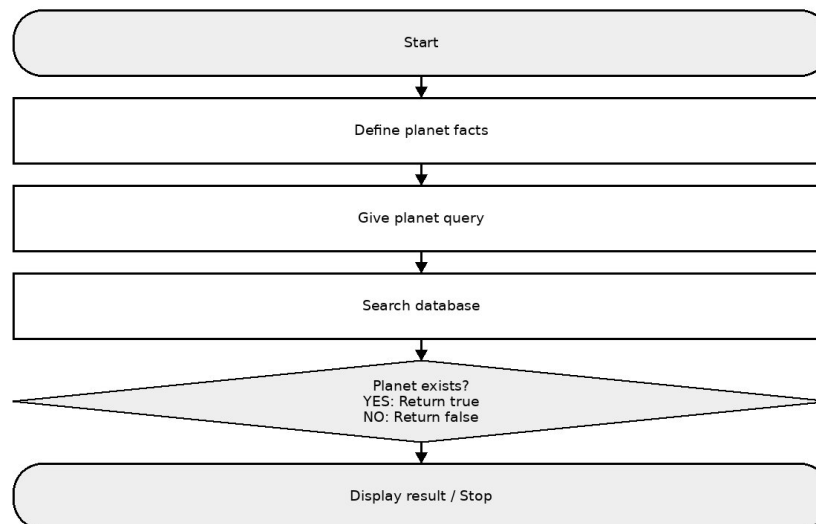
Algorithm

1. Start the program.
2. Define facts for the planets.
3. Store the planet names using the planet predicate.
4. Use a query to check whether a planet exists.
5. Display the result.
6. Stop.

Flow Chart

20: Prolog Program for Planets Database

Flow Chart - Prolog database



Program

```
planet(mercury).
```

```
planet(venus).
```

```
planet(earth).
```

```
planet(mars).
```

```
planet(jupiter).
```

```
planet(saturn).
```

```
planet(uranus).
```

```
planet(neptune).
```

```
is_planet(X) :-
```

```
    planet(X).
```

Output

```
?- is_planet(earth).
```

```
true.
```

```
?- is_planet(mars).
```

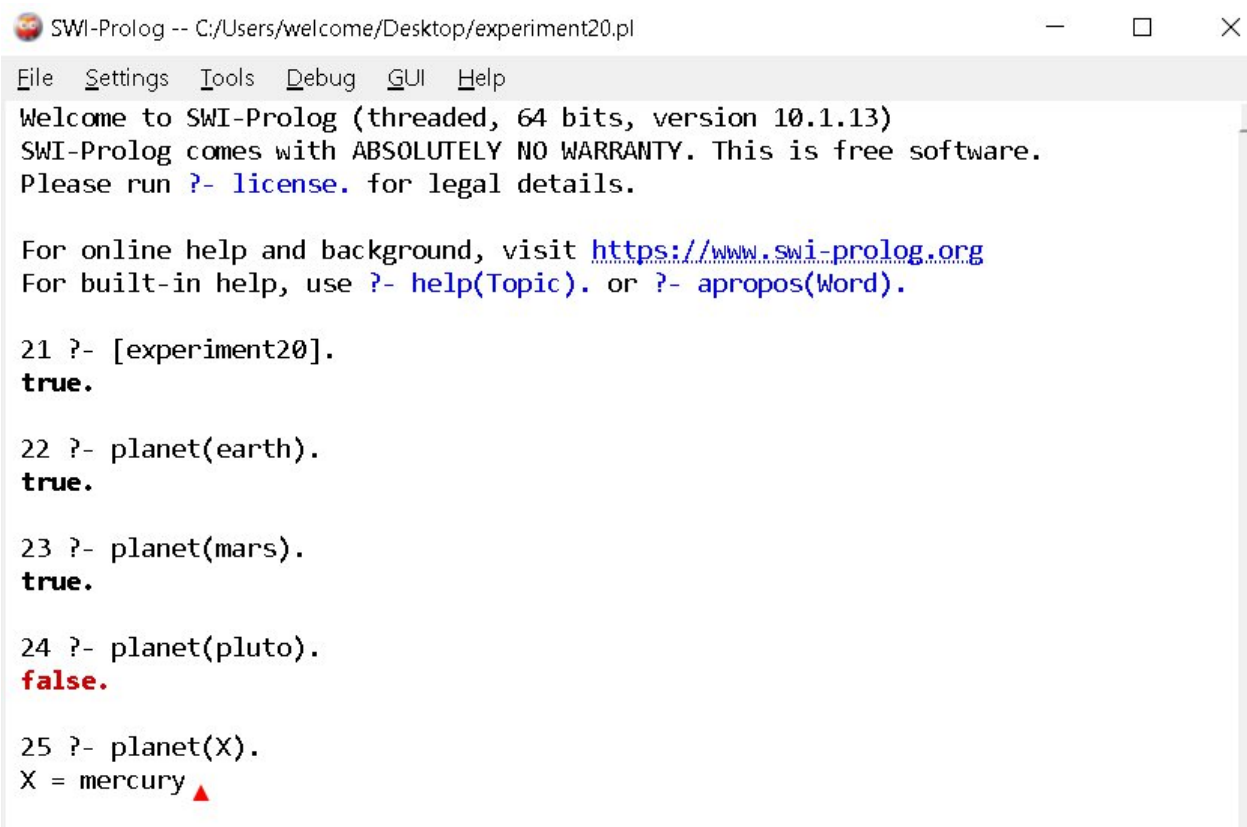
```
true.
```

```
?- is_planet(pluto).
```

```
false.
```

Result

Thus, the Prolog program was successfully executed to create and query the planets database.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment20.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

21 ?- [experiment20].
true.

22 ?- planet(earth).
true.

23 ?- planet(mars).
true.

24 ?- planet(pluto).
false.

25 ?- planet(X).
X = mercury ▲
```

21: Prolog Program to Implement Towers of Hanoi

Aim

To write and execute a Prolog program to solve the **Towers of Hanoi** problem using recursion.

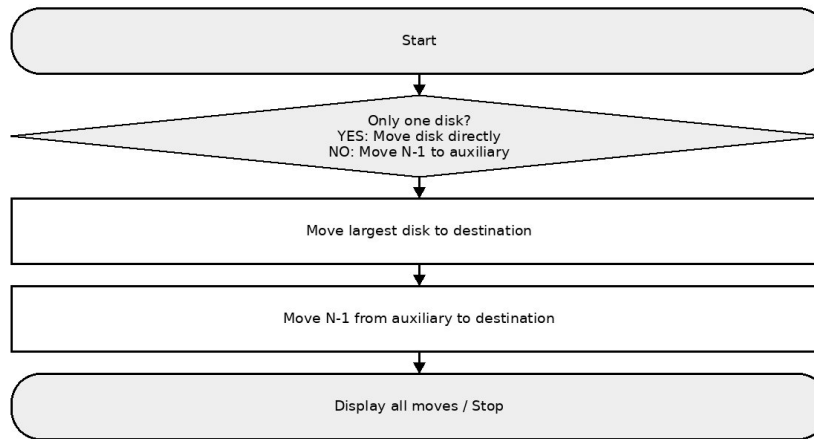
Algorithm

1. Start the program.
2. If there is only one disk, move it directly from source to destination.
3. For N disks, move N-1 disks from source to auxiliary peg.
4. Move the largest disk from source to destination.
5. Move N-1 disks from auxiliary peg to destination.
6. Repeat recursively until all disks are moved.
7. Display each move.
8. Stop.

Flow Chart

21: Prolog Program to Implement Towers of Hanoi

Flow Chart - Recursion in Prolog



Program

hanoi(1, Source, Destination, _) :-

write('Move disk from '),

write(Source),

write(' to '),

write(Destination),

nl.

hanoi(N, Source, Destination, Auxiliary) :-

N > 1,

N1 is N - 1,

hanoi(N1, Source, Auxiliary, Destination),

hanoi(1, Source, Destination, Auxiliary),

hanoi(N1, Auxiliary, Destination, Source).

Output

Query:

?- hanoi(3, left, right, middle).

Output:

Move disk from left to right

Move disk from left to middle

Move disk from right to middle

Move disk from left to right

Move disk from middle to left

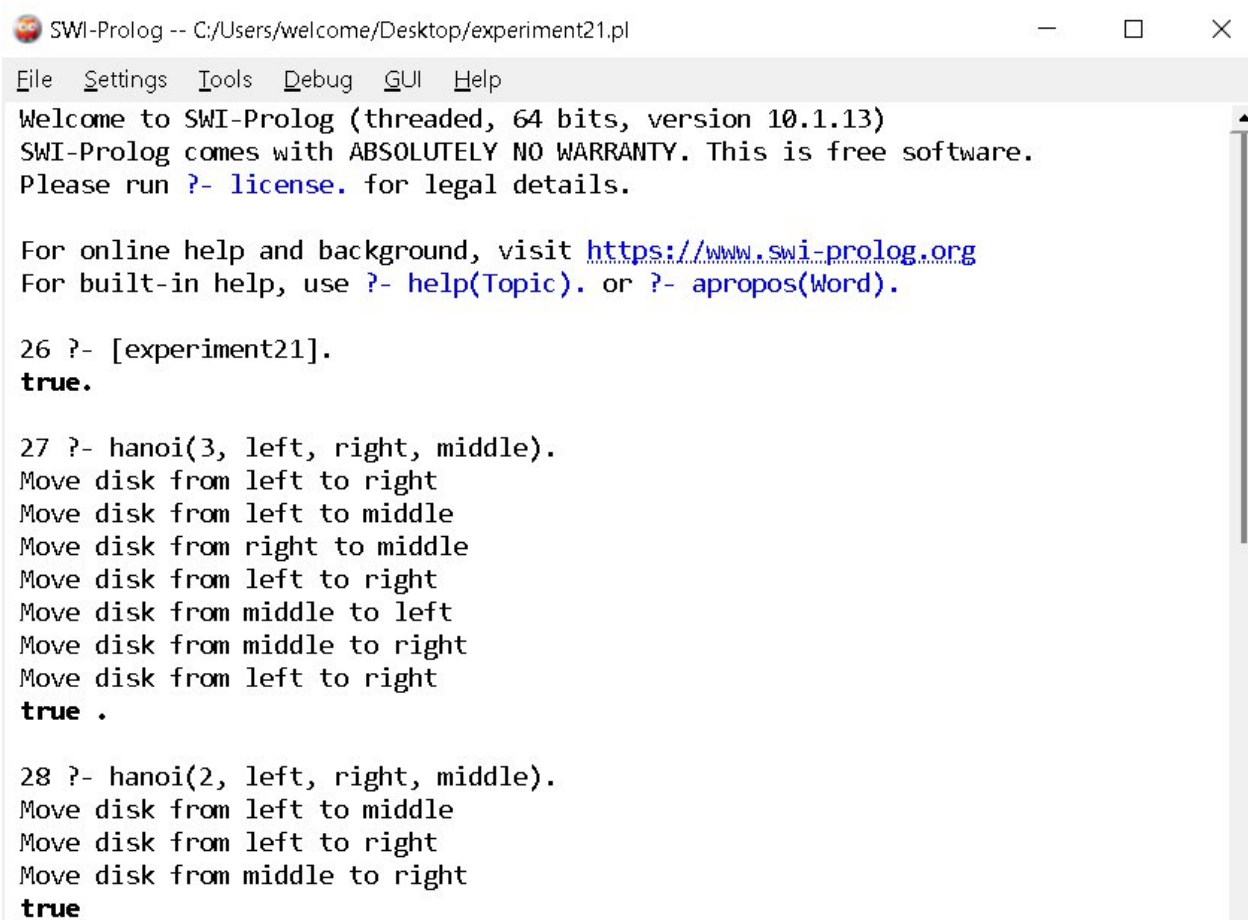
Move disk from middle to right

Move disk from left to right

true.

Result

Thus, the Prolog program was successfully executed to implement the **Towers of Hanoi** using recursion.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment21.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

26 ?- [experiment21].
true.

27 ?- hanoi(3, left, right, middle).
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right
true .

28 ?- hanoi(2, left, right, middle).
Move disk from left to middle
Move disk from left to right
Move disk from middle to right
true
```

21: Towers of Hanoi

Aim

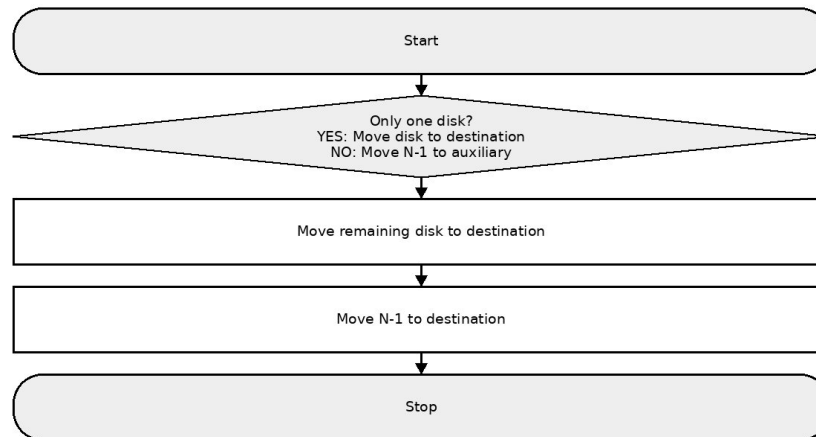
To write and execute a Prolog program to implement the Towers of Hanoi problem using recursion.

Algorithm

1. Start the program.
2. If there is one disk, move it directly to the destination.
3. Move $N-1$ disks from source to auxiliary.
4. Move the remaining disk from source to destination.
5. Move $N-1$ disks from auxiliary to destination.
6. Repeat recursively until all disks are moved.
7. Stop.

Flow Chart

21: Towers of Hanoi
Flow Chart - Recursion in Prolog



Program

hanoi(1, Source, Destination, _) :-

```
    write('Move disk from '),  
    write(Source),  
    write(' to '),  
    write(Destination),  
    nl.
```

hanoi(N, Source, Destination, Auxiliary) :-

$N > 1$,

$N1$ is $N - 1$,

hanoi($N1$, Source, Auxiliary, Destination),

hanoi(1, Source, Destination, Auxiliary),

hanoi($N1$, Auxiliary, Destination, Source).

Output

?- hanoi(3, left, right, middle).

Move disk from left to right

Move disk from left to middle

Move disk from right to middle

Move disk from left to right

Move disk from middle to left

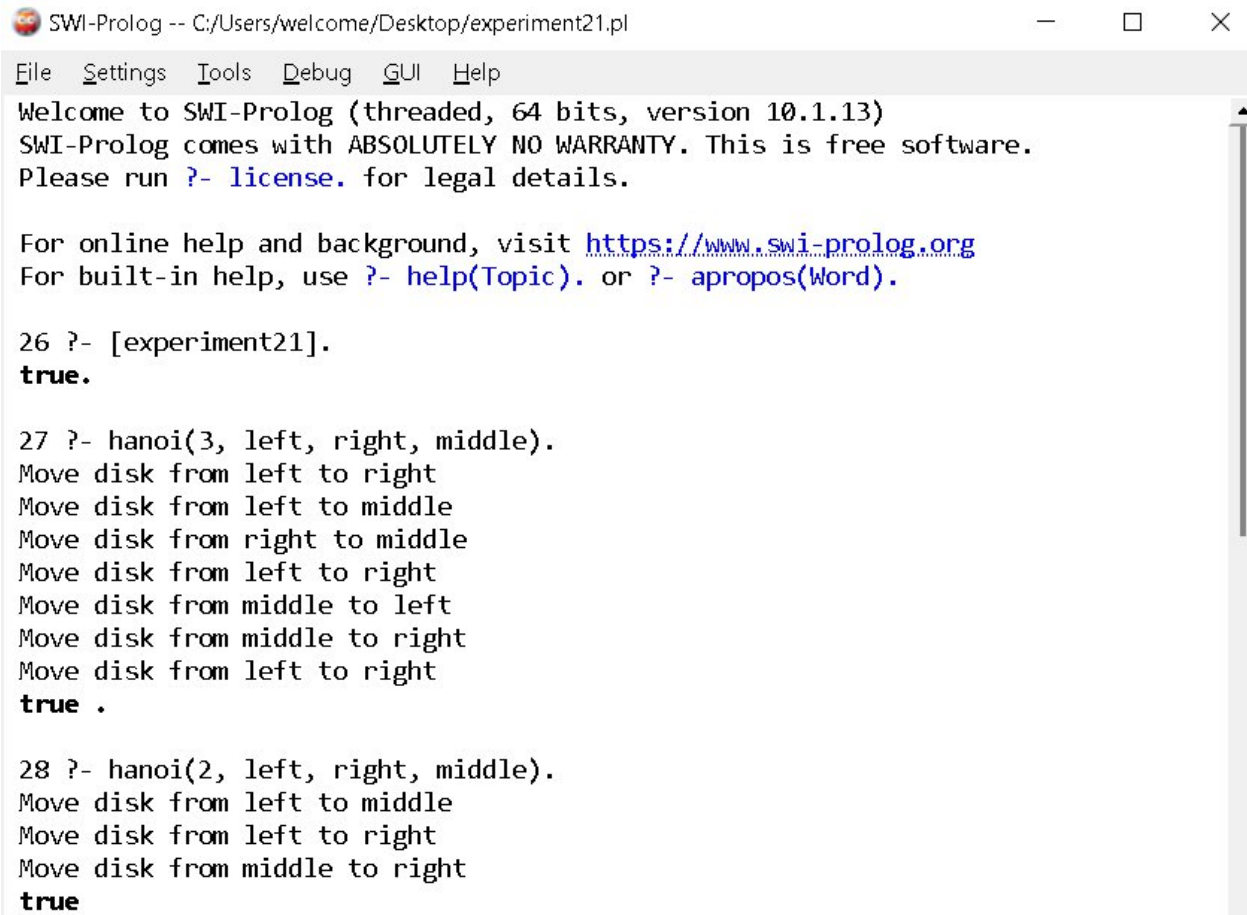
Move disk from middle to right

Move disk from left to right

true.

Result

Thus, the Prolog program was successfully executed to implement the Towers of Hanoi problem.



SWI-Prolog -- C:/Users/welcome/Desktop/experiment21.pl

File Settings Tools Debug GUI Help

Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run `?- license.` for legal details.

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

26 `?- [experiment21].`
true.

27 `?- hanoi(3, left, right, middle).`
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right
true .

28 `?- hanoi(2, left, right, middle).`
Move disk from left to middle
Move disk from left to right
Move disk from middle to right
true

22: Prolog Program to Check Whether a Bird Can Fly

Aim

To write and execute a Prolog program to determine whether a particular bird can fly using facts and rules.

Algorithm

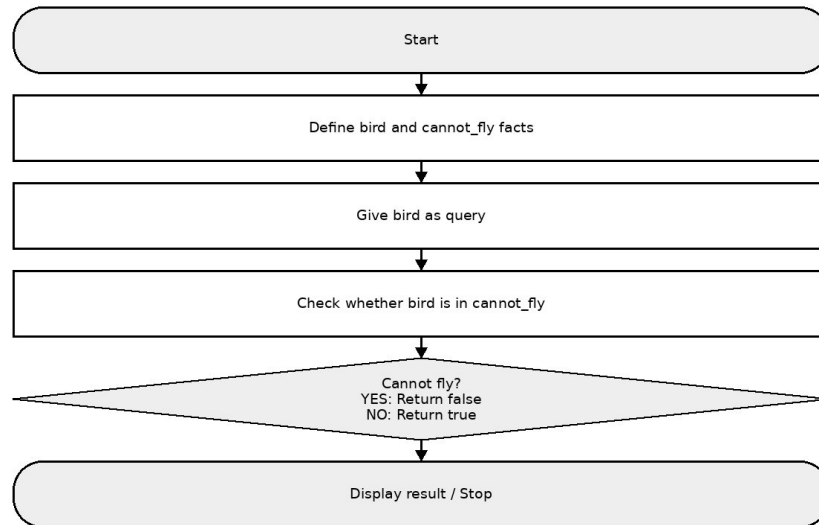
1. Start the program.

2. Define facts for birds.
3. Define birds that cannot fly.
4. Create a rule to determine whether a bird can fly.
5. Give the required query.
6. Display the result.
7. Stop.

Flow Chart

22: Prolog Program to Check Whether a Bird Can Fly

Flow Chart - Facts, rules and negation



Program

```
bird(parrot).
```

```
bird(eagle).
```

```
bird(pigeon).
```

```
bird(ostrich).
```

```
bird(penguin).
```

```
cannot_fly(ostrich).
```

```
cannot_fly(penguin).
```

```
can_fly(Bird) :-
```

```
    bird(Bird),
```

```
    \+ cannot_fly(Bird).
```

Output

?- can_fly(parrot).

true.

?- can_fly(eagle).

true.

?- can_fly(penguin).

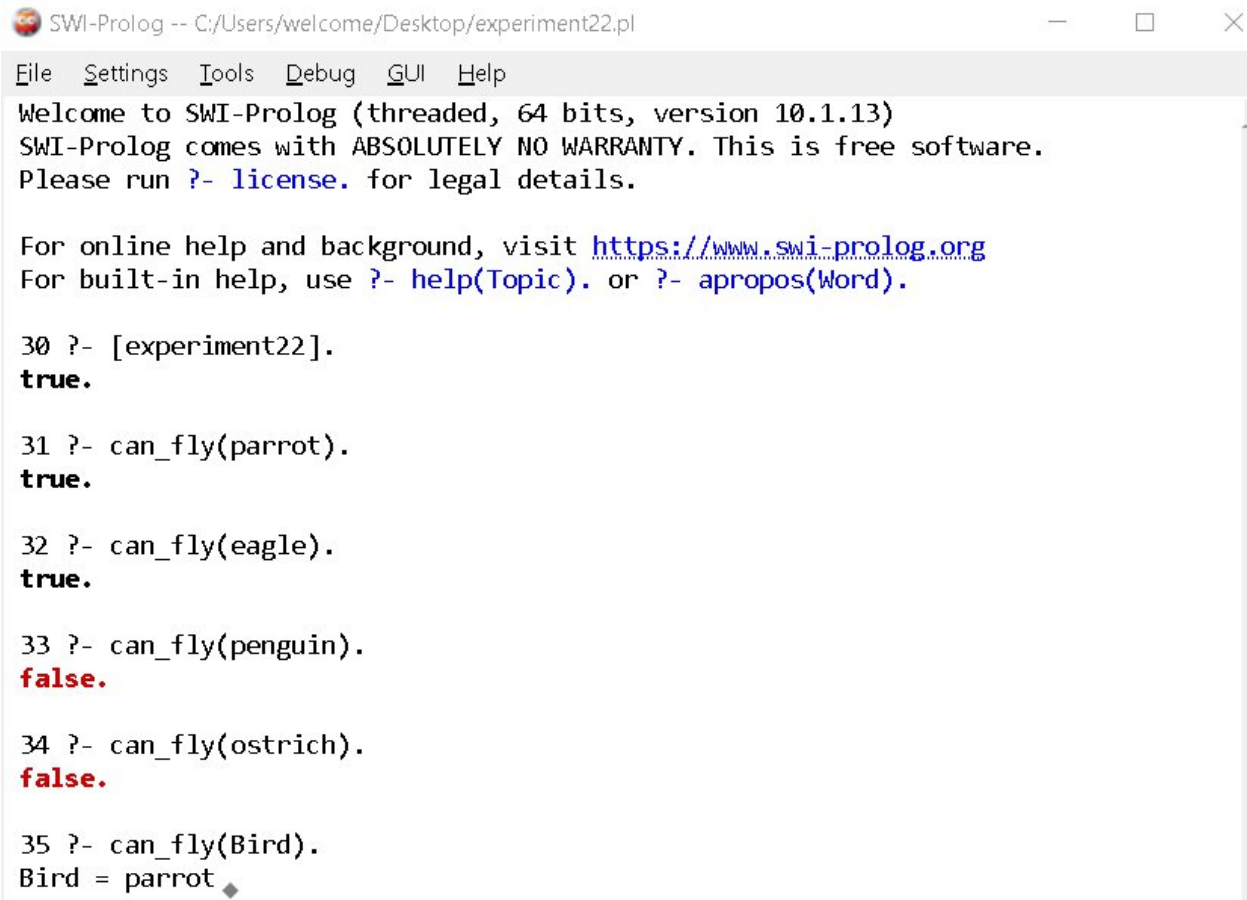
false.

?- can_fly(ostrich).

false.

Result

Thus, the Prolog program was successfully executed to determine whether a particular bird can fly or not.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment22.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

30 ?- [experiment22].
true.

31 ?- can_fly(parrot).
true.

32 ?- can_fly(eagle).
true.

33 ?- can_fly(penguin).
false.

34 ?- can_fly(ostrich).
false.

35 ?- can_fly(Bird).
Bird = parrot
```

23: Prolog Program to Implement Family Tree

Aim

To write and execute a Prolog program to represent a family tree and find relationships between family members.

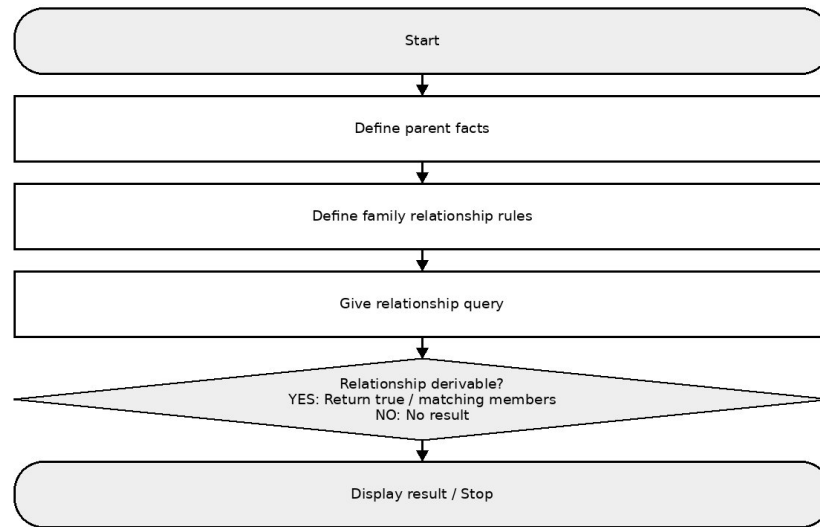
Algorithm

1. Start the program.
2. Define facts for parent relationships.
3. Define rules for father, mother, grandparent, and sibling relationships.
4. Use queries to find relationships.
5. Display the results.
6. Stop.

Flow Chart

23: Prolog Program to Implement Family Tree

Flow Chart - Prolog rules



Program

```
parent(john, mary).
```

```
parent(john, david).
```

```
parent(susan, mary).
```

```
parent(susan, david).
```

```
parent(david, tom).
```

```
parent(david, anna).
```

```
father(X, Y) :-
```

```
    parent(X, Y),
```

```
    male(X).
```

```
mother(X, Y) :-
```

```
    parent(X, Y),
```

```
    female(X).
```

```
male(john).
```

```
male(david).
```

```
male(tom).
```

```
female(susan).
```

```
female(mary).
```

female(anna).

grandparent(X, Z) :-

parent(X, Y),

parent(Y, Z).

sibling(X, Y) :-

parent(P, X),

parent(P, Y),

X \= Y.

Output

?- grandparent(john, tom).

true.

?- sibling(mary, david).

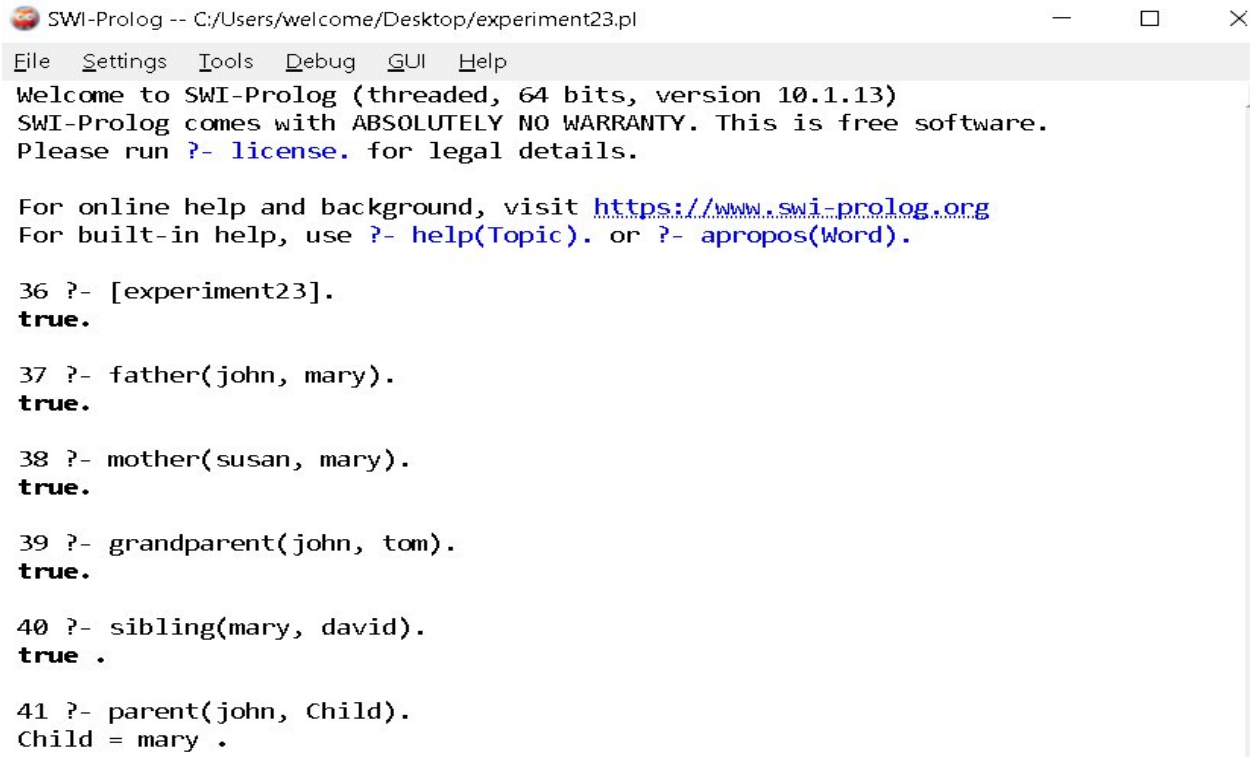
true.

?- parent(david, anna).

true.

Result

Thus, the Prolog program was successfully executed to implement a family tree and determine family relationships.



The screenshot shows a window titled "SWI-Prolog -- C:/Users/welcome/Desktop/experiment23.pl". The menu bar includes "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area contains the following content:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

36 ?- [experiment23].
true.

37 ?- father(john, mary).
true.

38 ?- mother(susan, mary).
true.

39 ?- grandparent(john, tom).
true.

40 ?- sibling(mary, david).
true .

41 ?- parent(john, Child).
Child = mary .
```

24: Prolog Program for Dieting System Based on Disease

Aim

To write and execute a Prolog program to suggest suitable foods based on a person's disease.

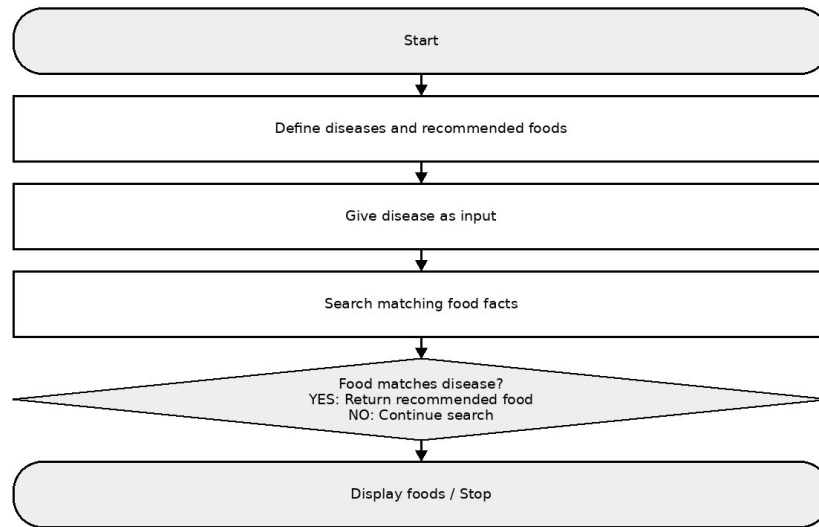
Algorithm

1. Start the program.
2. Define diseases and recommended foods.
3. Define rules to suggest food according to disease.
4. Give the disease as input.
5. Retrieve the recommended food.
6. Display the result.
7. Stop.

Flow Chart

24: Prolog Program for Dieting System Based on Disease

Flow Chart - Prolog knowledge base



Program

food(diabetes, vegetables).

food(diabetes, whole_grains).

food(diabetes, fruits).

food(hypertension, vegetables).

food(hypertension, banana).

food(hypertension, oats).

food(anemia, spinach).

food(anemia, beans).

food(anemia, apple).

suggest_food(Disease, Food) :-

food(Disease, Food).

Output

?- suggest_food(diabetes, Food).

Food = vegetables ;

Food = whole_grains ;

Food = fruits.

?- suggest_food(anemia, Food).

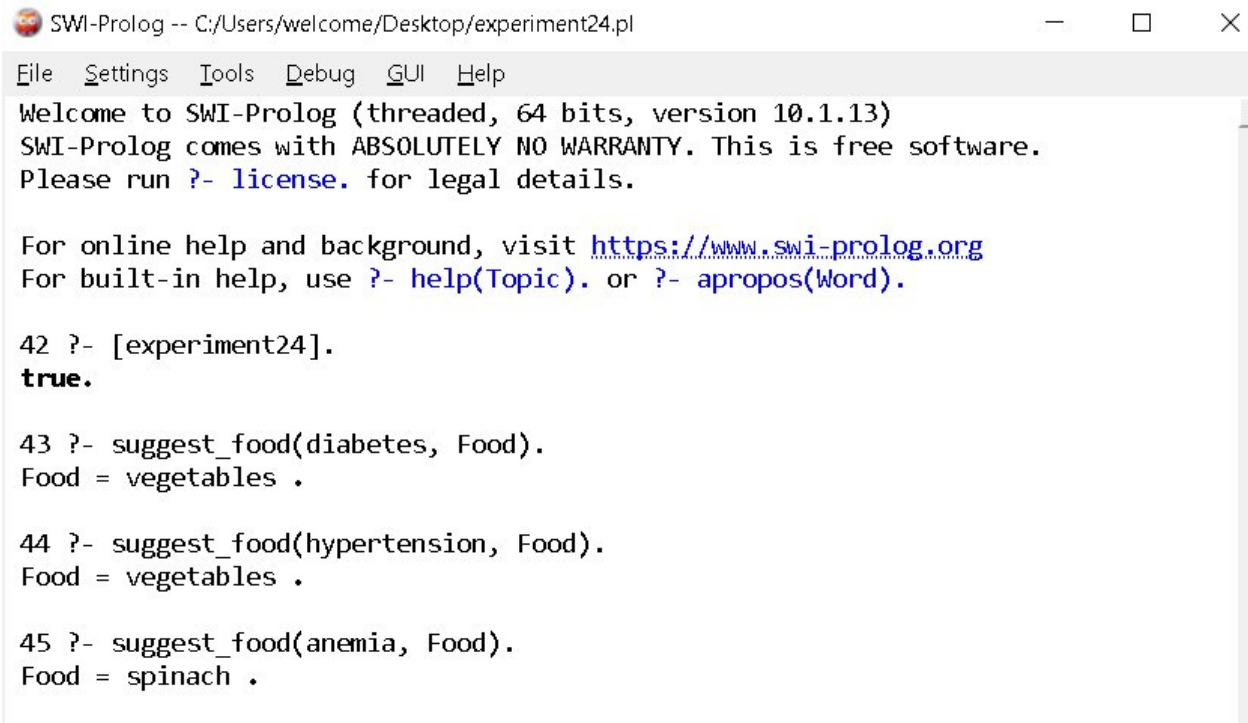
Food = spinach ;

Food = beans ;

Food = apple.

Result

Thus, the Prolog program was successfully executed to suggest foods based on the given disease.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment24.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

42 ?- [experiment24].
true.

43 ?- suggest_food(diabetes, Food).
Food = vegetables .

44 ?- suggest_food(hypertension, Food).
Food = vegetables .

45 ?- suggest_food(anemia, Food).
Food = spinach .
```

25: Prolog Program to Implement Monkey Banana Problem

Aim

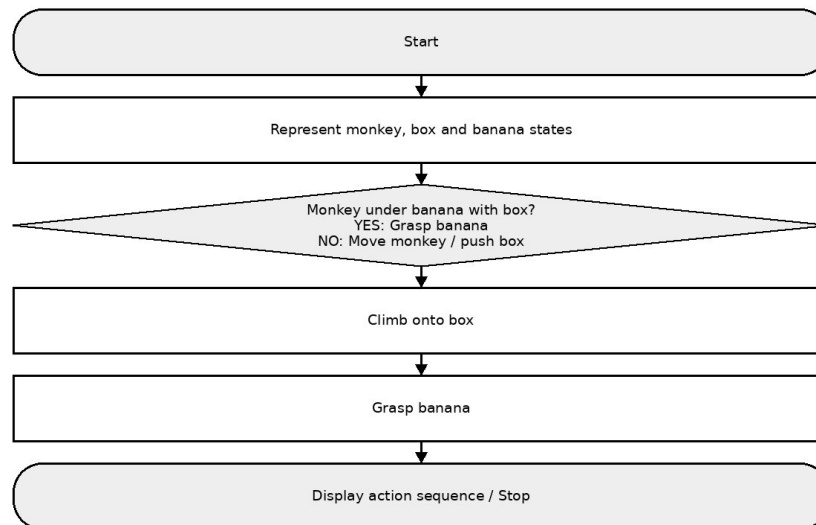
To write and execute a Prolog program to solve the Monkey Banana problem using logical reasoning and state transitions.

Algorithm

1. Start the program.
2. Represent the monkey, box, and banana positions as states.
3. Check whether the monkey is under the banana and has the box.
4. If the box is not under the banana, move the box.
5. Push the box to the banana position.
6. Climb onto the box.
7. Grasp the banana.
8. Display the solution.
9. Stop.

Flow Chart

25: Prolog Program to Implement Monkey Banana Problem
Flow Chart - State-space reasoning



Program

```
move(state(Monkey, Box, Banana, OnBox),  
state(Banana, Box, Banana, OnBox),
```

```
move_monkey) :-
```

```
Monkey \= Banana.
```

```
move(state(Monkey, Monkey, Banana, OnBox),
```

```
state(Banana, Banana, Banana, OnBox),
```

```
push_box) :-
```

```
Monkey \= Banana.
```

```
move(state(Monkey, Box, Banana, no),
```

```
state(Box, Box, Banana, yes),
```

```
climb) :-
```

```
Monkey = Box.
```

```
solve(state(Monkey, Box, Banana, no), [Action]) :-
```

```
Monkey = Banana,
```

```
Box = Banana,
```

```
Action = grasp_banana.
```

```
solve(State, [Action|Actions]) :-
```

```
move(State, NewState, Action),
```

```
solve(NewState, Actions).
```

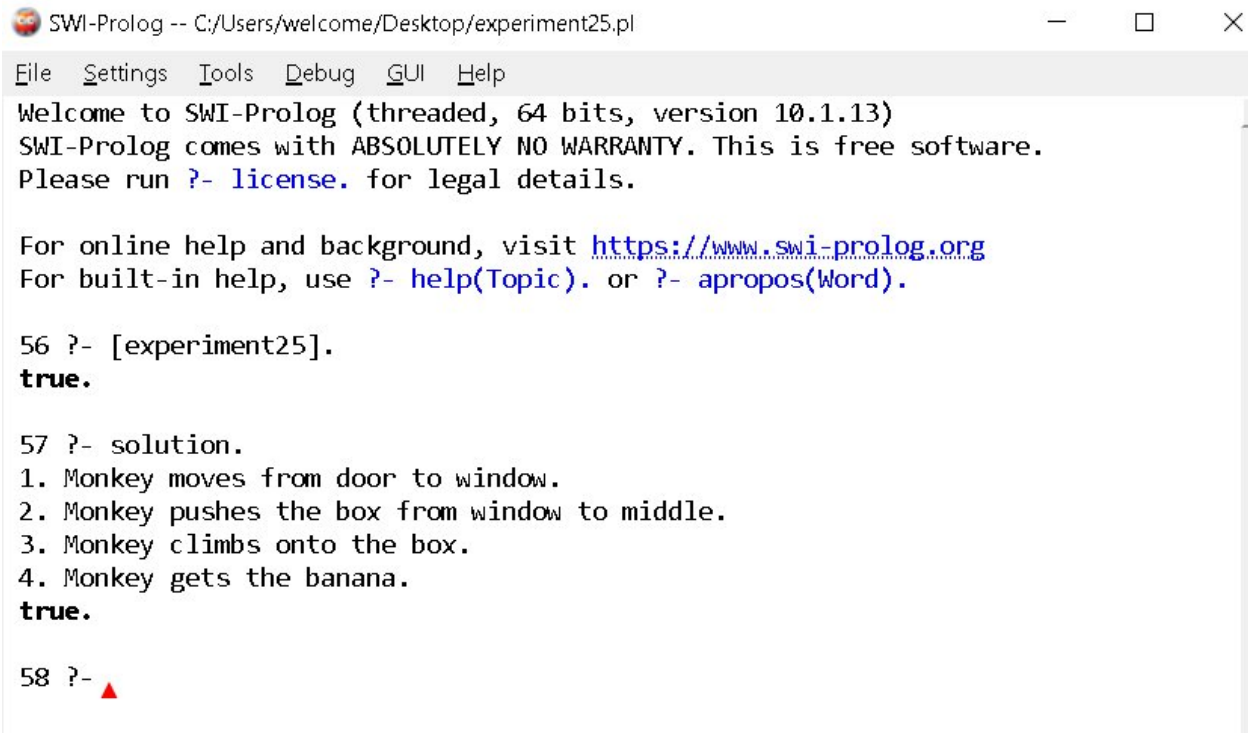
Output

```
?- solve(state(door, middle, middle, no), Actions).
```

```
Actions = [move_monkey, push_box, climb, grasp_banana].
```

Result

Thus, the Prolog program was successfully executed to implement the Monkey Banana problem using state-space reasoning.



The screenshot shows a window titled "SWI-Prolog -- C:/Users/welcome/Desktop/experiment25.pl". The menu bar includes File, Settings, Tools, Debug, GUI, and Help. The main text area displays the following content:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

56 ?- [experiment25].
true.

57 ?- solution.
1. Monkey moves from door to window.
2. Monkey pushes the box from window to middle.
3. Monkey climbs onto the box.
4. Monkey gets the banana.
true.

58 ?- ▲
```

26: Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write and execute a Prolog program to identify the color of fruits using backtracking.

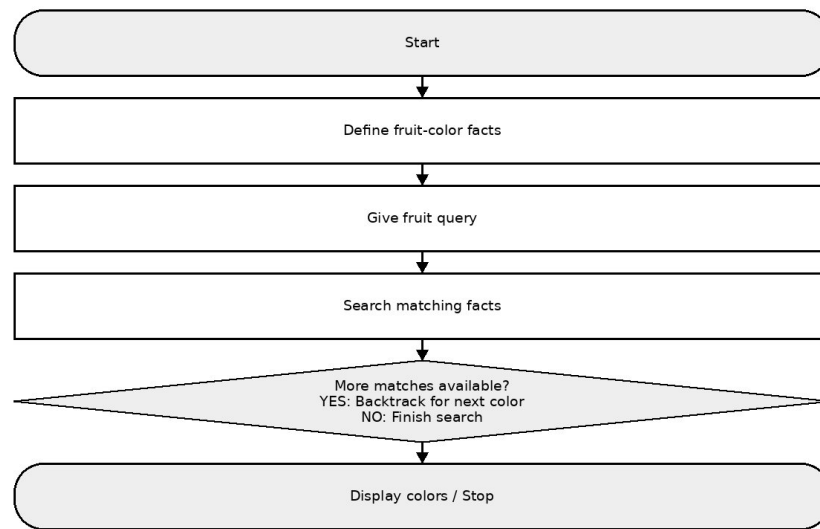
Algorithm

1. Start the program.
2. Define facts containing fruits and their colors.
3. Create a rule to identify the color of a fruit.
4. Give a fruit as input.
5. Prolog searches the database for matching facts.
6. Use backtracking to find additional solutions when available.
7. Display the result.
8. Stop.

Flow Chart

26: Prolog Program for Fruit and Its Color Using Backtracking

Flow Chart - Backtracking



Program

```
fruit_color(apple, red).
```

```
fruit_color(apple, green).
```

```
fruit_color(banana, yellow).
```

```
fruit_color(orange, orange).
```

```
fruit_color(grapes, green).
```

```
fruit_color(grapes, purple).
```

```
fruit_color(mango, yellow).
```

```
fruit_color_query(Fruit, Color) :-
```

```
    fruit_color(Fruit, Color).
```

Output

```
?- fruit_color_query(apple, Color).
```

```
Color = red ;
```

```
Color = green ;
```

```
false.
```

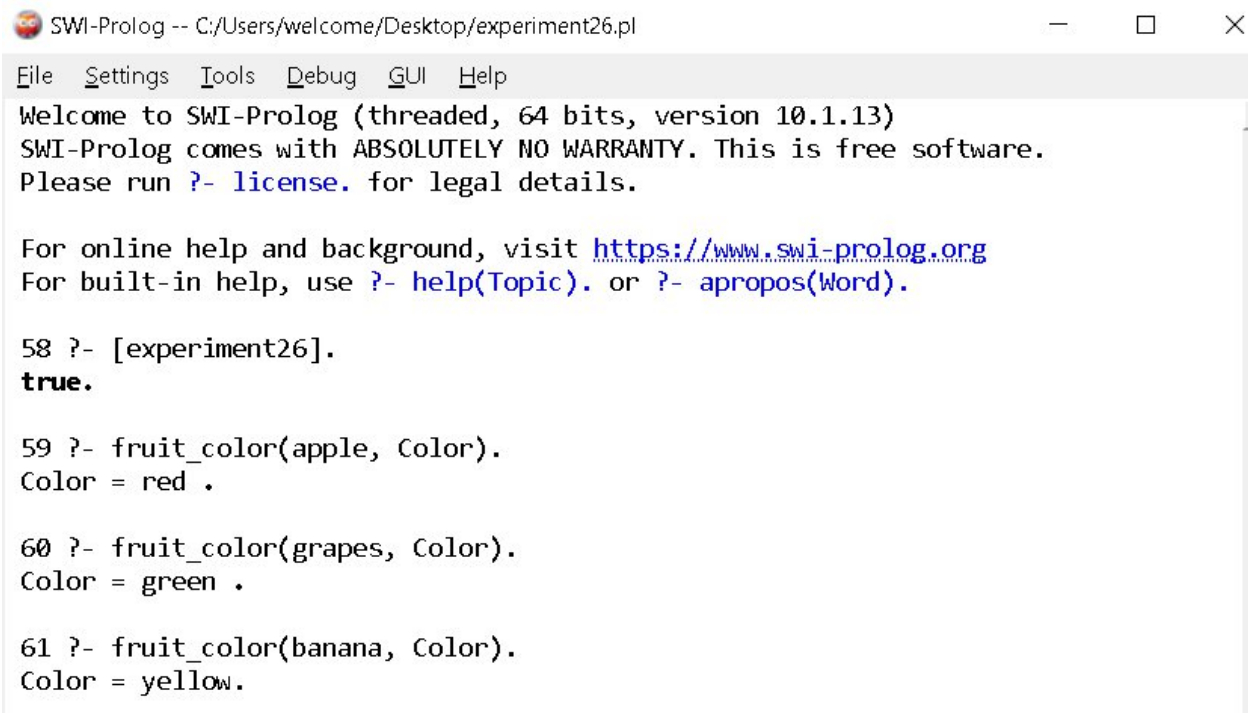
Another query:

```
?- fruit_color_query(grapes, Color).
```

Color = green ;
Color = purple ;
false.

Result

Thus, the Prolog program was successfully executed to identify fruit colors using **backtracking**



The screenshot shows the SWI-Prolog GUI window titled "SWI-Prolog -- C:/Users/welcome/Desktop/experiment26.pl". The window has a menu bar with "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content:

```
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

58 ?- [experiment26].
true.

59 ?- fruit_color(apple, Color).
Color = red .

60 ?- fruit_color(grapes, Color).
Color = green .

61 ?- fruit_color(banana, Color).
Color = yellow.
```

27: Prolog Program to Implement Best First Search Algorithm

Aim

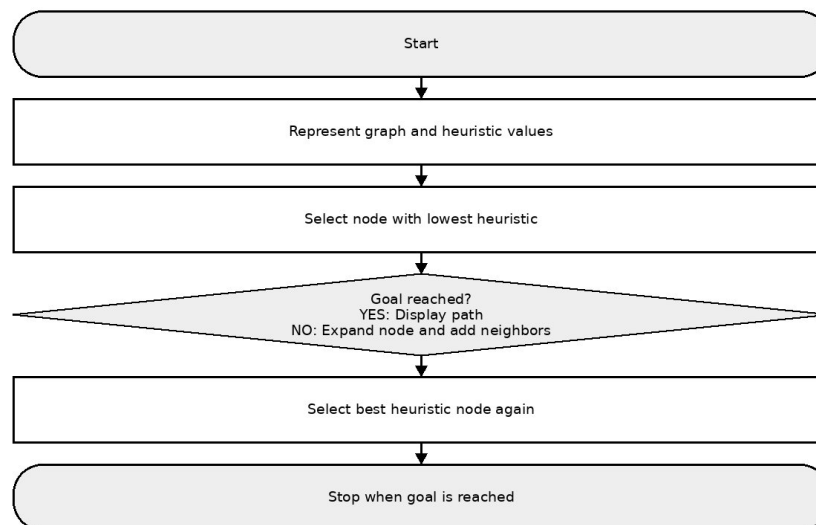
To write and execute a Prolog program to implement the Best First Search algorithm.

Algorithm

1. Start the program.
2. Represent the graph using nodes and heuristic values.
3. Select the node with the lowest heuristic value.
4. Expand the selected node.
5. Add its neighboring nodes to the open list.
6. Continue selecting the node with the best heuristic value.
7. Stop when the goal node is reached.
8. Display the path.

Flow Chart

27: Prolog Program to Implement Best First Search Algorithm
Flow Chart - Best First Search



Program

```
edge(a, b).  
edge(a, c).  
edge(b, d).  
edge(b, e).  
edge(c, f).
```

```
edge(c, g).  
edge(d, h).  
edge(e, h).  
edge(f, h).  
edge(g, h).
```

```
heuristic(a, 10).  
heuristic(b, 6).  
heuristic(c, 5).  
heuristic(d, 4).  
heuristic(e, 3).  
heuristic(f, 2).  
heuristic(g, 1).  
heuristic(h, 0).
```

```
best_first(Start, Goal, Path) :-  
    search([Start], Goal, [Start], RevPath),  
    reverse(RevPath, Path).
```

```
search([Goal|_], Goal, _, [Goal]).
```

```
search([Current|Open], Goal, Visited, [Current|Path]) :-  
    findall(H-Next,  
        (edge(Current, Next),  
         \+ member(Next, Visited),  
         heuristic(Next, H)),  
        Children),  
    keysort(Children, Sorted),  
    pairs_values(Sorted, Nodes),  
    append(Nodes, Open, NewOpen),  
    NewOpen = [Next|_],  
    search(NewOpen, Goal, [Next|Visited], Path).
```

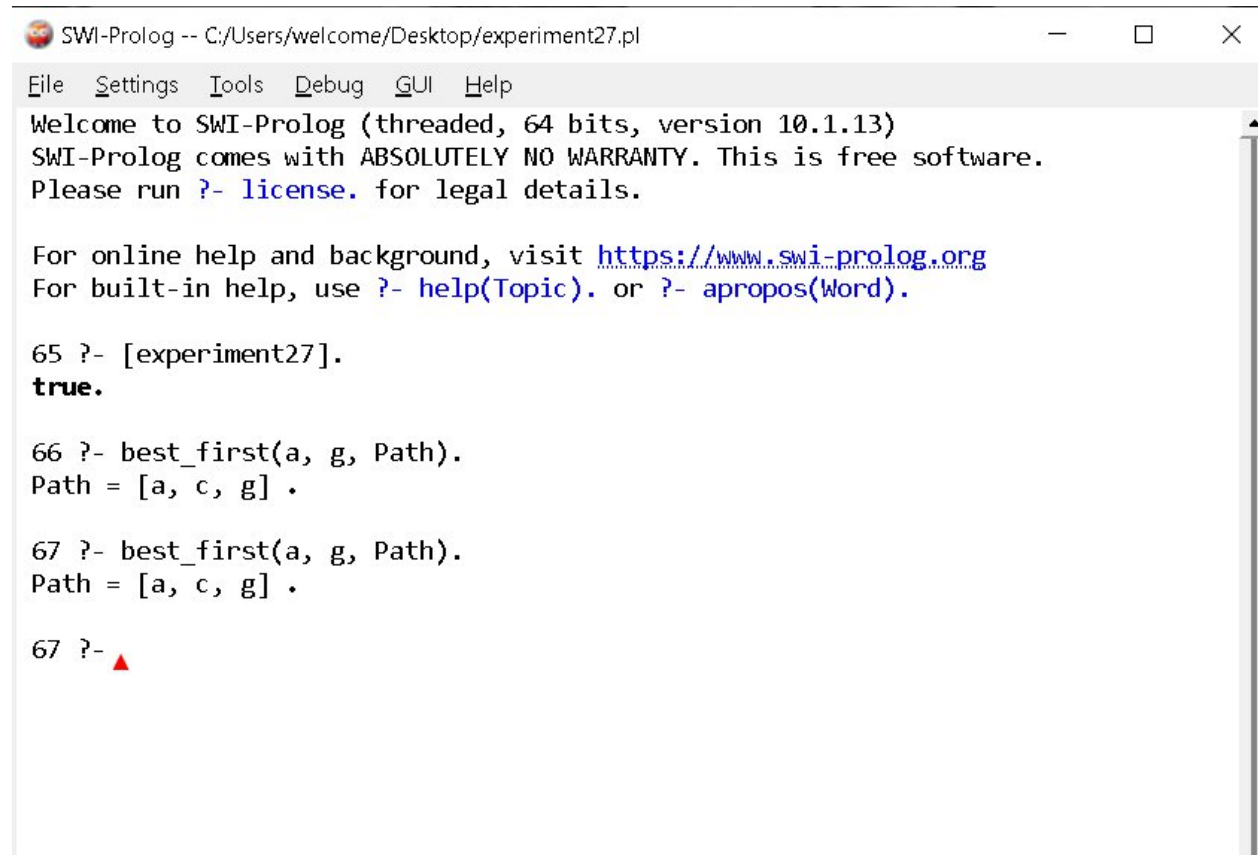

Output

```
?- best_first(a, h, Path).
```

```
Path = [a, c, f, h].
```

Result

Thus, the Prolog program was successfully executed to implement the **Best First Search algorithm**.

A screenshot of the SWI-Prolog graphical user interface. The window title is "SWI-Prolog -- C:/Users/welcome/Desktop/experiment27.pl". The menu bar includes "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content: a welcome message for SWI-Prolog (threaded, 64 bits, version 10.1.13), a disclaimer about warranty, and instructions for online and built-in help. Below this, the execution of a Prolog program is shown. Line 65 shows the query "[experiment27]." being executed, resulting in "true.". Line 66 shows the query "best_first(a, g, Path).", resulting in the path "Path = [a, c, g] .". Line 67 shows the query "best_first(a, g, Path).", also resulting in "Path = [a, c, g] .". The prompt "67 ?-" is followed by a red triangle cursor.

```
SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

65 ?- [experiment27].
true.

66 ?- best_first(a, g, Path).
Path = [a, c, g] .

67 ?- best_first(a, g, Path).
Path = [a, c, g] .

67 ?- ▲
```

28: Prolog Program for Medical Diagnosis

Aim

To write and execute a Prolog program to diagnose a disease based on the symptoms provided.

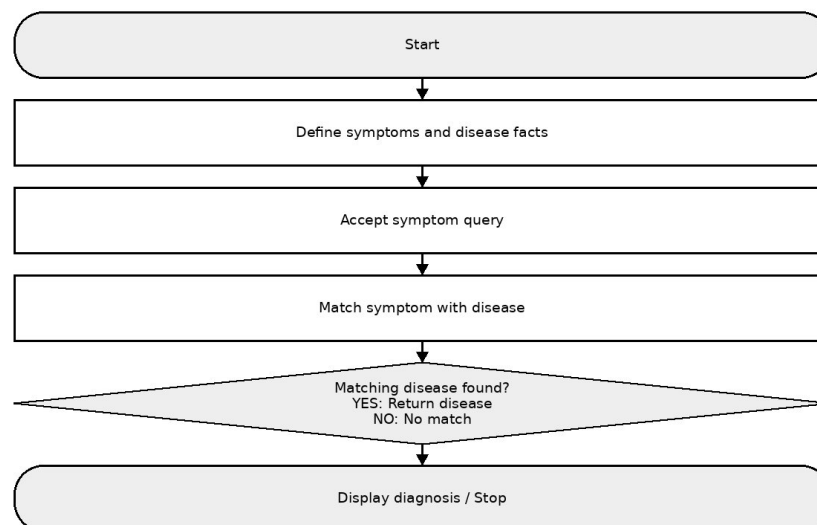
Algorithm

1. Start the program.
2. Define symptoms associated with different diseases.
3. Accept symptoms as input.
4. Compare the given symptoms with the stored facts.
5. Identify the matching disease.
6. Display the diagnosis.
7. Stop.

Flow Chart

28: Prolog Program for Medical Diagnosis

Flow Chart - Rule-based diagnosis



Program

```
symptom(fever, flu).
```

```
symptom(cough, flu).
```

```
symptom(body_pain, flu).
```

```
symptom(fever, malaria).
```

```
symptom(chills, malaria).
```

```
symptom(headache, malaria).
```

```
symptom(cough, cold).
```

```
symptom(sneezing, cold).
```

```
symptom(runny_nose, cold).
```

```
diagnosis(Disease, Symptom) :-
```

```
    symptom(Symptom, Disease).
```

Output

```
?- diagnosis(flu, cough).
```

```
true.
```

```
?- diagnosis(malaria, chills).
```

```
true.
```

```
?- diagnosis(cold, fever).
```

```
false.
```

Result

Thus, the Prolog program was successfully executed to identify possible diseases based on the given symptoms.

File Settings Tools Debug GUI Help

Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run `?- license.` for legal details.

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

```
68 ?- [experiment28].  
true.
```

```
68 ?- diagnosis(flu, cough).  
true.
```

```
69 ?- diagnosis(malaria, chills).  
true.
```

```
70 ?- diagnosis(Disease, fever).  
Disease = flu .
```

29: Prolog Program for Forward Chaining

Aim

To write and execute a Prolog program to implement forward chaining using facts and rules.

Algorithm

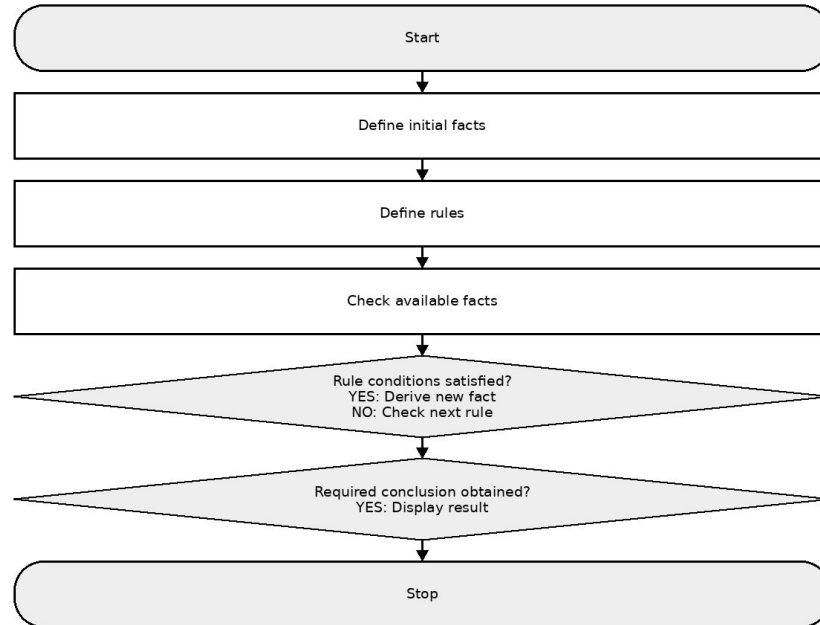
1. Start the program.
2. Define the initial facts.
3. Define rules based on the facts.

4. Check the available facts.
5. Apply rules whose conditions are satisfied.
6. Derive new facts.
7. Continue until the required conclusion is obtained.
8. Display the result.

Flow Chart

29: Prolog Program for Forward Chaining

Flow Chart - Forward chaining



Program

```
fact(sunny).
```

```
fact(warm).
```

```
rule(good_weather) :-
```

```
    fact(sunny),
```

```
    fact(warm).
```

```
rule(go_outside) :-
```

```
    rule(good_weather).
```

```
forward_chaining(X) :-
```

```
    fact(X).
```

```
forward_chaining(X) :-  
    rule(X).
```

Output

```
?- forward_chaining(good_weather).
```

```
true.
```

```
?- forward_chaining(go_outside).
```

```
true.
```

Result

Thus, the Prolog program was successfully executed to demonstrate **Forward Chaining**.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment29.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

68 ?- [experiment29].
true.

69 ?- forward_chaining(good_weather).
true.

70 ?- forward_chaining(go_outside).
true.

71 ?- forward_chaining(sunny).
true ▲
```

30: Prolog Program for Backward Chaining

Aim

To write and execute a Prolog program to implement backward chaining using facts and rules.

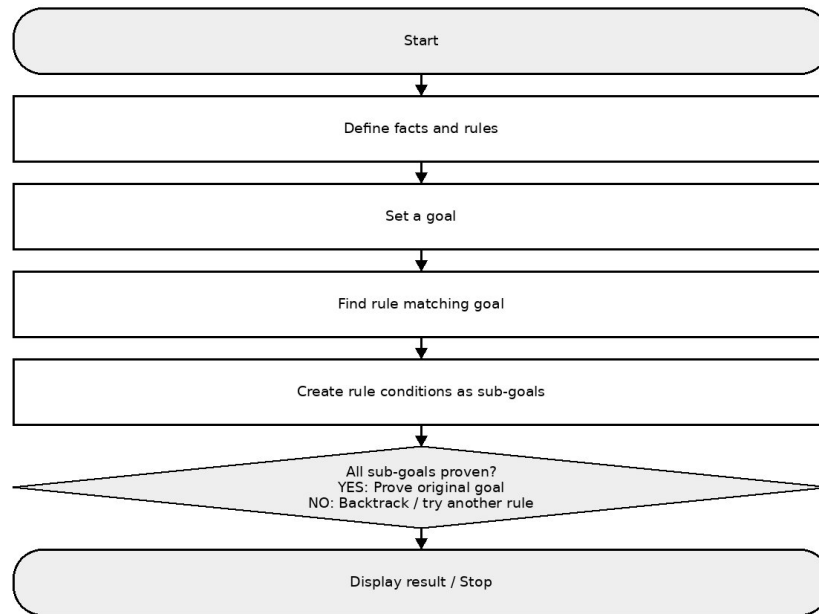
Algorithm

1. Start the program.
2. Define facts and rules.
3. Set a goal to be proved.
4. Search for a rule whose conclusion matches the goal.
5. Treat the rule conditions as sub-goals.
6. Check whether the sub-goals can be proved using available facts or rules.
7. If all sub-goals are satisfied, prove the original goal.
8. Display the result.

Flow Chart

30: Prolog Program for Backward Chaining

Flow Chart - Backward chaining



Program

```
fact(sunny).
```

```
fact(warm).
```

```
good_weather :-  
    fact(sunny),  
    fact(warm).
```

```
go_outside :-  
    good_weather.
```

Output

```
?- good_weather.
```

```
true.
```

```
?- go_outside.
```

```
true.
```



```
?- rainy.
```

```
false.
```

Result

Thus, the Prolog program was successfully executed to demonstrate **Backward Chaining**.

A screenshot of the SWI-Prolog graphical user interface. The window title is "SWI-Prolog -- C:/Users/welcome/Desktop/experiment30.pl". The menu bar includes "File", "Settings", "Tools", "Debug", "GUI", and "Help". The main text area displays the following content: a welcome message for version 10.1.13, a disclaimer about no warranty, and instructions on how to get help. Below this, three Prolog queries are shown with their results: "?- [experiment30]. true.", "?- good_weather. true.", and "?- go_outside. true.". The text is in a monospaced font, with some elements in blue to indicate hyperlinks or user input.

```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment30.pl
File Settings Tools Debug GUI Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.1.13)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

68 ?- [experiment30].
true.

69 ?- good_weather.
true.

70 ?- go_outside.
true.
```

31: Create a Web Blog Using WordPress

Aim

To create a web blog using WordPress and demonstrate HTML elements such as Anchor Tag, Title Tag, headings, paragraphs, and images.

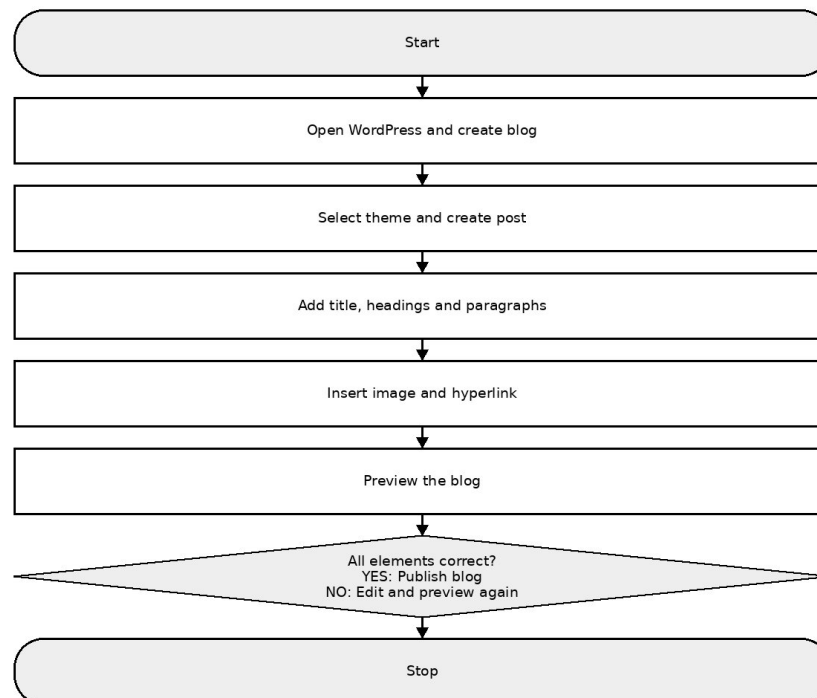
Algorithm

1. Start WordPress.
2. Create a new website or blog.
3. Select a suitable theme.
4. Create a new blog post.
5. Add a suitable title using the Title Tag.
6. Add headings and paragraph content.
7. Insert an image into the blog.
8. Create a hyperlink using the Anchor Tag.
9. Preview and publish the blog.
10. Verify that all elements are displayed correctly.

Flow Chart

31: Create a Web Blog Using WordPress

Flow Chart - WordPress / HTML elements



Program / HTML Example

```
<!DOCTYPE html>

<html>

<head>

    <title>My Web Blog</title>

</head>


<body>


    <h1>Welcome to My Web Blog</h1>


    <h2>About My Blog</h2>


    <p>

        This blog contains information about technology and programming.

    </p>


    <a href="https://www.google.com">Visit Google</a>


    <br><br>


</body>

</html>
```

Output

Welcome to My Web Blog

About My Blog

This blog contains information about technology
and programming.

Visit Google

[Blog Image]

Result

Thus, a web blog was successfully created using WordPress, demonstrating the **Title Tag**, **Heading Tags**, **Paragraph Tag**, **Anchor Tag**, and **Image Tag**.



```
SWI-Prolog -- C:/Users/welcome/Desktop/experiment31.pl
File Settings Tools Debug GUI Help
73 ?- [experiment31].
true.

74 ?- display.
Blog: my_blog
Title: My Web Blog
Heading: Welcome to My Web Blog
Paragraph: This blog contains information about technology and programming.
Link: Visit Google
URL: https://www.google.com
Image: blog.jpg
true.
```